



ESCUELA DE INGENIERÍA DE FUENLABRADA

GRADO EN INGENIERÍA DE ROBÓTICA SOFTWARE

TRABAJO FIN DE GRADO

**Diseño e implementación de un módulo de replanificación basado en
modelos de lenguaje para PlanSys2**

Autor: Álvaro Valencia Maiquez

Tutor: Francisco Martín Rico

Cotutor: Francisco Miguel Moreno Olivo

Curso académico 2025 / 2026

©2026 Álvaro Valencia

Algunos derechos reservados

Este documento se distribuye bajo la licencia “Atribución 4.0 Internacional”
de Creative Commons,

disponible en: <https://creativecommons.org/licenses/by/4.0/deed.es>

Resumen

Este Trabajo Fin de Grado presenta el diseño, implementación y validación de un módulo complementario para sistemas de planificación robótica basados en ROS 2 y PlanSys2. El objetivo principal del trabajo es incorporar mecanismos de replanificación asistida por modelos de lenguaje pequeños ejecutados localmente, permitiendo que un sistema robótico adapte su plan ante cambios del entorno, eventos de percepción o fallos durante la ejecución sin delegar la planificación formal en el modelo.

El módulo propuesto se ha desarrollado como un submódulo software independiente, descargable y compilable, con el propósito de facilitar su integración tanto con PlanSys2 como con otros sistemas que requieran capacidades de replanificación apoyadas en modelos locales. La solución aborda la generación y gestión de contexto, la comunicación con el *backend* de inferencia, el procesamiento de sus respuestas y la validación de las modificaciones propuestas antes de incorporarlas al flujo de planificación.

La validación se realiza mediante un demostrador robótico y una comparativa incremental entre modelos de lenguaje pequeños, analizando la calidad de la replanificación, la robustez del sistema y las limitaciones observadas. Los resultados muestran que, en un escenario controlado de corrección de estado PDDL, estos modelos pueden generar propuestas válidas bajo un formato estructurado, aunque su comportamiento depende del caso de uso, del *backend* y del hardware empleado. Además, la reutilización de KV-cache reduce de forma significativa el coste del procesamiento del *prompt* en hardware embarcado.

Abstract

This Bachelor Thesis presents the design, implementation, and validation of a complementary module for robotic planning systems based on ROS 2 and PlanSys2. The main goal of the project is to incorporate replanning mechanisms assisted by small language models executed locally, enabling a robotic system to adapt its plan when environmental changes, perception events, or execution failures occur, without delegating formal planning to the model.

The proposed module has been developed as an independent software submodule that can be downloaded and compiled, with the purpose of facilitating its integration with PlanSys2 as well as with other systems requiring replanning capabilities supported by local models. The solution addresses context generation and management, communication with the inference backend, response processing, and validation of the proposed modifications before incorporating them into the planning workflow.

The system is validated through a robotic demonstrator and an incremental comparison between small language models, analysing replanning quality, system robustness, and observed limitations. The results show that, in a controlled PDDL state-correction scenario, these models can generate valid proposals under a structured output format, although their behaviour depends on the use case, the backend, and the hardware platform. In addition, KV-cache reuse significantly reduces prompt-processing cost on embedded hardware.

Índice general

Resumen	I
Abstract	II
1. Introducción	1
1.1. Motivación	1
1.2. Problema abordado	2
1.3. Propuesta del TFG	2
1.4. Estructura de la memoria	3
2. Objetivos y metodología	4
2.1. Objetivos específicos	4
2.2. Requisitos funcionales	5
2.3. Requisitos no funcionales	5
2.4. Metodología de desarrollo	6
2.5. Criterios de validación	6
3. Marco tecnológico y fundamentos de planificación: ROS 2, PlanSys2 y LLMs	7
3.1. ROS 2 como infraestructura de integración robótica	8
3.1.1. Modelo basado en nodos y comunicación distribuida	8
3.1.2. Papel de ROS 2 en la arquitectura desarrollada	9
3.2. Fundamentos de planificación automática en robótica	10
3.2.1. Estados, acciones y objetivos	10
3.2.2. Dominio y problema de planificación	11

3.2.3.	Mundo cerrado frente a mundo abierto	11
3.2.4.	Planificación simbólica y ejecución robótica	12
3.3.	PlanSys2 como sistema de planificación simbólica	12
3.3.1.	Componentes principales de PlanSys2	13
3.3.2.	Gestión del dominio y del problema	13
3.3.3.	Generación y ejecución de planes	14
3.3.4.	POPF como planificador externo	15
3.3.5.	Limitaciones ante información incompleta	15
3.4.	Replanificación en sistemas robóticos	16
3.4.1.	Necesidad de replanificar	16
3.4.2.	Fallos de ejecución y cambios en el entorno	16
3.4.3.	Actualización del problema de planificación	17
3.4.4.	Dificultades de la replanificación en entornos abiertos	17
3.5.	Uso de LLMs como apoyo a la replanificación simbólica	18
3.5.1.	Motivación para introducir LLMs	18
3.5.2.	Construcción del contexto de entrada	19
3.5.3.	Interpretación de eventos, observaciones y trazas	20
3.5.4.	Generación de propuestas de actualización	21
3.5.5.	Formato estructurado de salida	21
3.5.6.	Validación antes de modificar el problema	22
3.5.7.	Relación entre LLM, PlanSys2 y POPF	22
3.6.	Estado del arte	23
3.7.	Limitaciones y riesgos del uso de LLMs	24
3.7.1.	Alucinaciones e información no verificable	24
3.7.2.	Inconsistencias con el estado real del sistema	25
3.7.3.	Dependencia del prompt	25
3.7.4.	Variabilidad entre ejecuciones	25
3.7.5.	Necesidad de validación externa	26
3.8.	Síntesis del capítulo	27

4. Diseño del módulo propuesto	29
4.1. Visión general del sistema	29
4.2. Arquitectura del módulo	30
4.3. Integración con PlanSys2	31
4.4. Flujo de información del sistema	31
4.4.1. Flujo del nodo integrador	32
4.5. Gestión de eventos, contexto y estado	33
4.6. Interfaz con modelos LLM	33
4.7. Decisiones de diseño y alternativas descartadas	34
5. Implementación	35
5.1. Estructura del repositorio/submódulo	35
5.2. Componentes principales desarrollados	36
5.3. Integración y compilación	37
5.4. Generación de prompts y control del contexto	37
5.4.1. Control del contexto: resumen del registro de acciones	37
5.4.2. Plantilla del prompt	38
5.4.3. Esquema JSON de salida	39
5.4.4. Orden del prompt y reutilización de la KV-cache	40
5.4.5. Contribución a llama_ros: reutilización de KV-cache	41
5.5. Procesamiento de respuestas del LLM	42
5.5.1. Extracción tolerante del bloque JSON	42
5.5.2. Validación estructural y traducción al mensaje ROS 2	43
5.6. Validación de propuestas antes de la replanificación	44
5.7. Gestión de errores y casos límite	44
5.7.1. Errores en la respuesta del modelo	45
5.7.2. Errores operacionales: <i>timeouts</i> y cancelación	45
5.7.3. Errores de configuración y de infraestructura	46
5.7.4. Casos límite contemplados	47
5.8. Ejemplo de uso del módulo	47

6. Experimentación y validación	50
6.1. Escenario de prueba y caso de uso	50
6.2. Metodología experimental	51
6.3. Modelos comparados	52
6.4. Resultados con prompts sintéticos (V2)	52
6.5. Validación con prompt real (V3)	55
6.6. Despliegue sin GPU	57
6.7. Viabilidad en Raspberry Pi 5	58
6.8. Reutilización de KV-cache: validación empírica del contrato de orden	59
6.9. Limitaciones de la validación	60
7. Impacto y responsabilidades	62
7.1. Impacto social	62
7.2. Impacto económico	62
7.3. Impacto medioambiental	63
7.4. Responsabilidad ética y profesional	63
7.5. Uso responsable de LLMs en sistemas robóticos	63
8. Conclusiones y trabajo futuro	64
8.1. Consecución de objetivos	64
8.2. Conclusiones técnicas	67
8.3. Limitaciones del sistema	68
8.4. Validez y escalabilidad del enfoque local	68
8.5. Competencias del grado aplicadas	68
8.6. Nuevas competencias adquiridas	69
8.7. Líneas de trabajo futuro	69
Bibliografía	70
A. Repositorio del proyecto	72

Índice de figuras

3.1. Contrato de entrada y salida de POPF.	14
3.2. Ciclo de replanificación asistida por LLM.	18
4.1. Visión general del sistema	29
4.2. Flujo de replanificación	32
5.1. Ciclo de replanificación	49
6.1. Frontera de Pareto score–latencia (V2).	53
6.2. Score por modelo, escenario y modo de log (V2).	54
6.3. Distribución de modos de fallo por modelo (V2).	54
6.4. Cambio de score V2 a V3 por modelo.	56
6.5. Frontera de Pareto con el prompt real (V3).	56
6.6. Score GPU frente a CPU (V3).	58
6.7. Latencia GPU frente a CPU (V3).	58
6.8. Reutilización de KV-cache en Raspberry Pi 5.	60

Índice de tablas

2.1. Requisitos funcionales	5
2.2. Requisitos no funcionales	6
3.1. Mecanismos de comunicación de ROS 2 y casos típicos de uso.	9
3.2. Comparativa del papel del LLM en enfoques de planificación robótica asistida por modelos de lenguaje. La fila destacada corresponde a la propuesta de este trabajo.	24
3.3. Resumen de los riesgos del uso de LLMs en el módulo y mitigaciones aplicadas o delegadas.	27
5.1. Componentes del módulo	36
6.1. Casos de replanificación	51
6.2. Versiones experimentales	51
6.3. Métricas experimentales	52
6.4. Ranking V2	53
6.5. Ranking V3	55
6.6. GPU frente a CPU en V3	57
8.1. Objetivos y evidencias	65
8.2. Requisitos y evidencias	66
A.1. Repositorios del proyecto	72

Capítulo 1

Introducción

La robótica moderna combina percepción, planificación, control y ejecución para permitir que un sistema actúe de forma autónoma en entornos dinámicos. En este contexto, los sistemas de planificación automática permiten representar objetivos, acciones disponibles y estados del mundo para generar secuencias de acciones que conduzcan desde una situación inicial hasta una situación deseada.

Sin embargo, los entornos reales rara vez permanecen inmutables durante la ejecución de un plan. Pueden aparecer objetos inesperados, fallar acciones, cambiar la localización de elementos relevantes o recibirse nueva información procedente de sensores. Ante estas situaciones, el robot debe ser capaz de adaptar su comportamiento y generar un nuevo plan coherente con el estado actualizado del entorno.

1.1. Motivación

La motivación principal de este trabajo surge de la necesidad de dotar a los sistemas de planificación robótica de mecanismos más flexibles para interpretar eventos de ejecución y generar nuevas propuestas de planificación. En particular, los modelos de lenguaje ofrecen una capacidad interesante para procesar información contextual, resumir eventos, interpretar descripciones textuales y proponer modificaciones sobre el estado o los objetivos del sistema.

No obstante, el uso de modelos de lenguaje en robótica plantea riesgos importantes. Sus respuestas pueden ser ambiguas, inconsistentes o no verificables si se incorporan directamente al sistema. Por ello, resulta necesario diseñar una arquitectura que aproveche su capacidad de razonamiento contextual, pero manteniendo mecanismos de validación, trazabilidad y control sobre la información que se integra en el proceso de planificación.

Una restricción deliberada orienta este trabajo: el modelo de lenguaje debe ejecutarse localmente, sin depender de servicios remotos. Esta decisión no es accesorio, sino una premisa de diseño

que responde a varios motivos. En primer lugar, muchos despliegues robóticos reales operan en entornos sin conectividad fiable o con políticas de privacidad que impiden enviar información del entorno a infraestructura externa. En segundo lugar, la inferencia remota introduce coste por consulta y dependencia de un proveedor, lo que dificulta la reproducibilidad y compromete la viabilidad económica del despliegue. Por último, la ejecución local permite acotar el comportamiento del sistema sobre hardware controlado y medir su coste real de cómputo, aspecto que se analiza experimentalmente en el Capítulo 6. Esta restricción condiciona toda la cadena de diseño, desde la elección de modelos pequeños cuantizados hasta el contrato de orden del *prompt* que permite reutilizar la caché de claves-valores entre invocaciones.

1.2. Problema abordado

El problema abordado en este TFG consiste en diseñar e implementar un sistema software capaz de asistir procesos de replanificación en sistemas robóticos, como PlanSys2. El módulo debe permitir recopilar información relevante sobre el estado del sistema, eventos de ejecución y observaciones del entorno, construir un contexto adecuado para un modelo de lenguaje, procesar la respuesta generada y devolver una propuesta estructurada que pueda ser evaluada por un nodo integrador antes de solicitar una nueva planificación.

Una restricción adicional orienta el trabajo: el módulo se diseña pensando en modelos de lenguaje pequeños (*Small Language Models*, SLM), cuantizados y ejecutados localmente, sin depender de servicios externos ni de *fine-tuning*. Esta decisión condiciona tanto el diseño del *prompt* como la estrategia de especialización del modelo para cada caso de uso.

1.3. Propuesta del TFG

La propuesta de este trabajo es el desarrollo de un módulo software independiente que pueda integrarse con PlanSys2 o con otros sistemas basados en ROS 2 que requieran capacidades de replanificación asistidas por modelos de lenguaje. ROS 2 proporciona una arquitectura modular basada en nodos y mecanismos de comunicación como tópicos, servicios y acciones, lo que facilita la integración de componentes software en sistemas robóticos distribuidos.

El módulo se plantea como una capa externa, por lo que no requiere modificar los componentes internos de PlanSys2 y puede adaptarse a otros planificadores que proporcionen eventos de ejecución equivalentes.

La solución se plantea como un componente complementario, no como una sustitución del planificador clásico. De esta forma, el modelo de lenguaje actúa como una herramienta auxiliar para interpretar el contexto y proponer modificaciones, mientras que la aplicación integradora

mantiene la decisión final sobre si aplicar o no las propuestas generadas.

La propuesta se concreta sobre un modelo pequeño de propósito general ejecutado en local, especializado para cada despliegue a través del propio contexto de entrada y no mediante reentrenamiento. Esta decisión persigue reproducibilidad, independencia de proveedores externos y viabilidad sobre hardware con recursos limitados, aspectos que se analizan experimentalmente en el Capítulo 6.

1.4. Estructura de la memoria

La memoria se organiza en ocho capítulos principales:

- El Capítulo 2 presenta los objetivos, requisitos y metodología de desarrollo.
- El Capítulo 3 describe el marco tecnológico en el que se apoya el proyecto.
- El Capítulo 4 expone el diseño del módulo propuesto.
- El Capítulo 5 detalla la implementación realizada.
- El Capítulo 6 presenta la experimentación y validación del sistema.
- El Capítulo 7 analiza el impacto y las responsabilidades asociadas al trabajo.
- El Capítulo 8 recoge las conclusiones, las competencias aplicadas y las líneas de trabajo futuro.
- El Apéndice A recoge los repositorios públicos del proyecto, la licencia y la rama de referencia. Las instrucciones de instalación, compilación, ejemplos de ejecución y configuración experimental se mantienen en los README de los propios repositorios, que constituyen la documentación viva del código y se actualizan junto con él.

Capítulo 2

Objetivos y metodología

El objetivo general de este Trabajo Fin de Grado es diseñar, implementar y validar un módulo complementario e independiente, integrable con PlanSys2, que permita incorporar capacidades de asistencia a la replanificación mediante modelos de lenguaje, manteniendo una arquitectura modular, reproducible y verificable.

2.1. Objetivos específicos

Los objetivos específicos del trabajo son los siguientes:

- Analizar el funcionamiento de ROS 2, PlanSys2 y los mecanismos de planificación automática utilizados en el proyecto, identificando los elementos necesarios para integrar un módulo externo de asistencia a la replanificación.
- Diseñar una arquitectura modular que permita integrar modelos de lenguaje en un flujo de replanificación robótica, manteniendo separadas las responsabilidades de captura de contexto, inferencia, procesamiento de respuesta e integración.
- Implementar un submódulo independiente, descargable y compilable, que pueda utilizarse junto con PlanSys2 sin modificar sus componentes internos.
- Diseñar una interfaz cliente/servidor que permita solicitar propuestas de replanificación desde nodos externos, verificable mediante la invocación del monitor desde una aplicación integradora.
- Permitir la incorporación de distintos *backends* LLM mediante una arquitectura basada en plugins, verificable añadiendo o sustituyendo un *backend* sin modificar el núcleo del monitor.

- Definir mecanismos de construcción de contexto y generación de *prompts* a partir del estado del sistema y eventos de ejecución, comprobando que la salida generada mantiene un formato estructurado.
- Procesar y validar las respuestas generadas por los modelos de lenguaje antes de incorporarlas al proceso de planificación, distinguiendo entre validación estructural, validación semántica y decisión de aceptación.
- Evaluar el comportamiento del módulo mediante escenarios de prueba y comparativas entre distintos modelos LLM, sobre al menos un escenario robótico representativo y una comparativa entre modelos, según se detalla en el Capítulo 6.

2.2. Requisitos funcionales

Los requisitos funcionales describen las capacidades que debe proporcionar el sistema desarrollado.

Identificador	Descripción
RF-01	El sistema debe poder integrarse con un entorno basado en ROS 2 y PlanSys2.
RF-02	El sistema debe recopilar información relevante del estado, eventos o contexto de ejecución.
RF-03	El sistema debe construir prompts estructurados para consultar un modelo de lenguaje.
RF-04	El sistema debe procesar la respuesta generada por el LLM.
RF-05	El sistema debe validar la información generada antes de utilizarla en la replanificación.
RF-06	El sistema debe permitir ejecutar escenarios de prueba reproducibles.

Tabla 2.1: Requisitos funcionales

2.3. Requisitos no funcionales

Los requisitos no funcionales recogen propiedades de calidad del sistema.

Identificador	Descripción
RNF-01	El módulo no debe requerir modificaciones internas en PlanSys2 ni depender directamente de sus componentes internos de planificación.
RNF-02	El código debe ser escalable y estar organizado en componentes claramente diferenciados.
RNF-03	El sistema debe poder compilarse y ejecutarse siguiendo instrucciones reproducibles.
RNF-04	El sistema debe ser compatible con ROS 2 Jazzy y ROS 2 Rolling.
RNF-05	Las decisiones generadas por el LLM deben ser trazables y revisables.
RNF-06	El sistema debe contemplar fallos, respuestas inválidas o inconsistencias generadas por el LLM.

Tabla 2.2: Requisitos no funcionales

2.4. Metodología de desarrollo

El desarrollo del proyecto se ha llevado a cabo siguiendo una metodología incremental. En primer lugar, se analizaron las tecnologías base, el flujo de planificación, y la arquitectura modular de PlanSys2. Posteriormente, se diseñó un sistema para aislar la comunicación con los modelos de lenguaje y el usuario mediante el concepto cliente/servidor. A continuación, se implementaron los componentes principales del sistema y se realizaron pruebas progresivas sobre escenarios controlados.

Durante el desarrollo se emplearon herramientas de asistencia mediante inteligencia artificial como apoyo en tareas de revisión e investigación. Las decisiones técnicas, la implementación, la validación experimental y la interpretación de resultados fueron realizadas por el autor.

2.5. Criterios de validación

La validación del proyecto se basa en comprobar que el módulo desarrollado cumple los objetivos planteados. Para ello se consideran criterios como la correcta integración con el entorno de ejecución, la capacidad de generar propuestas de replanificación, la robustez frente a respuestas inválidas del LLM, la reproducibilidad de los experimentos y la comparación entre diferentes modelos de lenguaje.

Capítulo 3

Marco tecnológico y fundamentos de planificación: ROS 2, PlanSys2 y LLMs

Este capítulo introduce las tecnologías y conceptos necesarios para comprender la arquitectura desarrollada. El sistema se apoya en tres pilares principales: ROS 2 como infraestructura de comunicación e integración robótica, PlanSys2 como sistema de planificación simbólica y los modelos de lenguaje como mecanismo auxiliar para interpretar información contextual durante la ejecución.

El objetivo de este capítulo no es describir de forma exhaustiva todas las capacidades de estas tecnologías, sino explicar aquellos aspectos que resultan relevantes para la propuesta planteada. Por ello, ROS 2 se presenta como el entorno que permite construir una arquitectura modular y distribuida; PlanSys2 se analiza como el marco que gestiona dominios, problemas, planes y ejecución; y los modelos de lenguaje se estudian desde el punto de vista de su utilidad para apoyar procesos de replanificación cuando el estado del mundo cambia o la información disponible es incompleta.

La motivación central del proyecto surge de una limitación habitual en sistemas de planificación simbólica aplicados a robótica: el planificador necesita trabajar sobre una representación formal, explícita y actualizada del mundo, mientras que un robot en ejecución recibe información dinámica, parcial y, en ocasiones, ambigua. En este contexto, los modelos de lenguaje no se utilizan para sustituir al planificador, sino como una herramienta auxiliar capaz de interpretar eventos, observaciones y trazas de ejecución para proponer posibles actualizaciones del problema de planificación.

3.1. ROS 2 como infraestructura de integración robótica

ROS 2 es el middleware robótico utilizado como base de comunicación e integración del sistema desarrollado [1]. Su función principal es proporcionar una infraestructura común sobre la que distintos componentes software puedan ejecutarse de forma independiente y comunicarse entre sí mediante interfaces bien definidas.

Un sistema robótico moderno suele estar compuesto por múltiples módulos con responsabilidades diferentes. Algunos componentes pueden encargarse de la percepción, otros de la planificación, otros de la ejecución de acciones y otros de la monitorización del estado. ROS 2 facilita esta separación mediante una arquitectura distribuida basada en nodos, donde cada nodo encapsula una funcionalidad concreta del sistema.

En la arquitectura desarrollada, ROS 2 permite que el módulo propuesto funcione como un componente independiente dentro de un sistema mayor. Esta decisión resulta importante porque evita modificar de forma intrusiva el sistema de planificación existente. El módulo puede comunicarse con otros nodos, recibir información de ejecución, enviar solicitudes y participar en el proceso de replanificación manteniendo una separación clara de responsabilidades.

3.1.1. Modelo basado en nodos y comunicación distribuida

El concepto central de ROS 2 es el nodo. Un nodo es una unidad de ejecución que realiza una tarea específica dentro del sistema robótico. Por ejemplo, un nodo puede publicar información procedente de sensores, otro puede procesar observaciones del entorno, otro puede gestionar el estado del problema de planificación y otro puede ejecutar una acción concreta del robot.

Esta organización permite dividir un sistema complejo en componentes más pequeños, reutilizables y sustituibles. Cada nodo puede desarrollarse y probarse de manera independiente, siempre que mantenga las interfaces de comunicación esperadas. Esto facilita la depuración del sistema y permite incorporar nuevos módulos sin rediseñar completamente la arquitectura.

ROS 2 proporciona varios mecanismos de comunicación entre nodos. Los tópicos permiten una comunicación basada en publicación y suscripción, adecuada para transmitir información continua o eventos del sistema. Los servicios permiten una comunicación de tipo petición-respuesta, útil para operaciones puntuales en las que un nodo solicita información o una acción concreta a otro componente. Las acciones, por su parte, están diseñadas para tareas de mayor duración, ya que permiten iniciar una operación, recibir realimentación durante su ejecución y obtener un resultado final.

Como resumen visual de lo anterior, la Tabla 3.1 recoge los tres mecanismos junto con su caso típico de uso.

Mecanismo	Semántica	Caso típico de uso
Tópico	Publicación / suscripción asíncrona; admite múltiples publicadores y suscriptores.	Difusión continua de datos sensoriales, odometría o eventos de estado.
Servicio	Petición / respuesta síncrona; un proveedor por nombre.	Consultas puntuales sobre el estado o la configuración de un nodo.
Acción	Petición / respuesta con realimentación, resultado final y posibilidad de cancelación.	Operaciones de larga duración como tareas de navegación o ejecución de un plan.

Tabla 3.1: Mecanismos de comunicación de ROS 2 y casos típicos de uso.

Estos mecanismos son relevantes para el proyecto porque permiten conectar el sistema de planificación simbólica con otros componentes del sistema robótico. La percepción, la ejecución de acciones, la monitorización de eventos y el módulo auxiliar basado en modelos de lenguaje pueden comunicarse mediante interfaces estándar de ROS 2, manteniendo una arquitectura modular.

3.1.2. Papel de ROS 2 en la arquitectura desarrollada

En este contexto, ROS 2 no constituye el objeto principal de estudio, sino la infraestructura que permite integrar los distintos componentes del sistema desarrollado. Su papel es proporcionar el entorno de ejecución, los mecanismos de comunicación y la organización modular necesaria para conectar el módulo propuesto con PlanSys2.

El módulo desarrollado se plantea como una capa complementaria dentro de un sistema de planificación existente. Esta capa recopila información relevante durante la ejecución, construye un contexto para el modelo de lenguaje y procesa la respuesta generada antes de que pueda influir en el problema de planificación. ROS 2 permite implementar esta lógica como parte de una arquitectura distribuida, en la que cada componente mantiene una responsabilidad concreta.

Además, el uso de ROS 2 facilita la reutilización del sistema en distintos escenarios robóticos. Al organizar el desarrollo en paquetes y nodos, el módulo puede integrarse en un espacio de trabajo ROS 2 junto con otros paquetes de percepción, planificación o ejecución. De esta forma, la propuesta no queda limitada a una aplicación monolítica, sino que puede adaptarse a diferentes configuraciones robóticas basadas en ROS 2.

3.2. Fundamentos de planificación automática en robótica

La planificación automática es una rama de la inteligencia artificial cuyo objetivo es generar una secuencia de acciones que permita alcanzar un objetivo a partir de un estado inicial. Para ello, el sistema necesita una representación formal del dominio, del estado actual y de las condiciones que se desean cumplir.

En robótica, la planificación automática permite que un robot razone sobre qué acciones debe realizar para completar una tarea. En lugar de programar manualmente una secuencia fija de pasos, se define un conjunto de acciones posibles y el planificador calcula una secuencia válida en función de la situación inicial y del objetivo definido.

Esta capacidad resulta especialmente útil cuando existen varias formas de resolver una tarea, cuando el entorno puede cambiar o cuando el sistema debe adaptarse a distintas situaciones. Sin embargo, su eficacia depende de que la representación del mundo utilizada por el planificador sea suficientemente precisa y esté actualizada.

3.2.1. Estados, acciones y objetivos

Un problema de planificación puede entenderse como la búsqueda de una secuencia de acciones que transforme un estado inicial en un estado que satisfaga ciertos objetivos. El estado describe qué hechos son verdaderos en un momento determinado. Estos hechos suelen representarse mediante predicados, que expresan propiedades de objetos o relaciones entre ellos.

Las acciones representan las operaciones que pueden modificar el estado del mundo. Cada acción suele definirse mediante parámetros, precondiciones y efectos. Los parámetros indican sobre qué objetos puede aplicarse la acción. Las precondiciones indican qué debe cumplirse para que la acción sea válida. Los efectos describen cómo cambia el estado tras su ejecución.

Los objetivos definen las condiciones que el sistema desea alcanzar. Un plan será válido si, al aplicar la secuencia de acciones desde el estado inicial, se obtiene un estado final que satisface dichos objetivos. Esta representación permite que el planificador busque soluciones de forma sistemática, basándose en reglas explícitas del dominio.

Por ejemplo, en un escenario robótico de transporte de objetos, una acción de recoger un libro podría requerir que el robot se encuentre en la misma ubicación que el libro y que el libro esté disponible. Como efecto, la acción podría indicar que el robot sostiene el libro y que este ya no se encuentra en la estantería. El planificador utiliza estas descripciones para decidir qué acciones son aplicables en cada momento y cómo se puede alcanzar el objetivo final.

3.2.2. Dominio y problema de planificación

En planificación simbólica suele distinguirse entre dominio y problema. El dominio describe las reglas generales del entorno: tipos de objetos, predicados, funciones y acciones disponibles. Es decir, define qué puede ocurrir dentro del sistema y bajo qué condiciones.

El problema, en cambio, describe una situación concreta dentro de ese dominio. Incluye los objetos existentes, el estado inicial y los objetivos que se quieren alcanzar. A partir de la descripción del dominio y del problema, el planificador intenta generar un plan válido.

Esta separación permite reutilizar un mismo dominio en múltiples situaciones. Por ejemplo, un dominio puede definir acciones generales como desplazarse, recoger un objeto o depositarlo en una ubicación. Después, diferentes problemas pueden describir distintos objetos, ubicaciones, estados iniciales y objetivos concretos.

En el contexto de la arquitectura propuesta, esta separación resulta fundamental. La replanificación puede entenderse como la necesidad de actualizar el problema de planificación cuando aparece nueva información durante la ejecución. El dominio puede mantenerse estable, pero el estado inicial, los objetos conocidos o incluso los objetivos pueden requerir modificaciones antes de solicitar un nuevo plan.

3.2.3. Mundo cerrado frente a mundo abierto

Un aspecto importante en planificación simbólica es la diferencia entre la suposición de mundo cerrado y la suposición de mundo abierto. En una suposición de mundo cerrado, el sistema considera que aquello que no está representado explícitamente como verdadero en el estado inicial no es cierto. Esta idea resulta útil para planificadores clásicos, ya que permite trabajar con una descripción concreta y finita del problema.

Sin embargo, los entornos robóticos reales se aproximan más a un mundo abierto. El robot puede no conocer todos los objetos presentes, puede recibir información incompleta de sus sensores o puede descubrir durante la ejecución que una suposición inicial era incorrecta. Por ejemplo, un objeto puede no encontrarse donde se esperaba, una acción puede fallar o un sensor puede detectar un elemento nuevo que no estaba incluido inicialmente en el problema.

Esta diferencia entre la representación cerrada utilizada por muchos planificadores y la naturaleza abierta del mundo real es una de las razones por las que la replanificación resulta necesaria. El sistema debe ser capaz de incorporar nueva información, actualizar su representación del estado y solicitar un nuevo plan cuando el plan anterior deja de ser válido.

Esta problemática aparece de forma directa en el sistema planteado. El planificador necesita trabajar con una representación formal del problema, pero el sistema robótico recibe eventos, observaciones y resultados de ejecución que pueden modificar la situación real. El módulo desa-

rollado busca actuar como una capa intermedia capaz de interpretar esa información contextual y proponer actualizaciones del problema antes de iniciar un nuevo proceso de planificación.

3.2.4. Planificación simbólica y ejecución robótica

La planificación simbólica permite representar acciones y estados mediante símbolos comprensibles a nivel lógico. Esta aproximación es útil en robótica porque permite razonar sobre tareas de alto nivel, como transportar objetos, visitar ubicaciones, inspeccionar zonas o completar secuencias de trabajo.

No obstante, la robótica introduce dificultades adicionales. Las acciones no son únicamente transformaciones lógicas de un estado, sino comportamientos que deben ejecutarse en un entorno físico o simulado. Por ello, una acción planificada puede fallar debido a obstáculos, errores de percepción, incertidumbre en la localización, problemas de control o cambios externos en el entorno.

Esta separación entre el plano simbólico y el plano físico obliga a diseñar mecanismos que conecten ambos niveles. El planificador genera una secuencia abstracta de acciones, pero el sistema de ejecución debe traducir cada acción en comportamientos concretos y detectar si el resultado coincide con lo esperado.

Cuando el resultado de una acción no coincide con lo previsto, el estado simbólico utilizado por el planificador puede dejar de representar correctamente la realidad. En ese caso, no basta con volver a ejecutar el mismo plan. Es necesario analizar qué ha ocurrido, actualizar el problema de planificación y generar un nuevo plan compatible con la situación actual.

3.3. PlanSys2 como sistema de planificación simbólica

PlanSys2 es un sistema de planificación para ROS 2 [2] que permite representar dominios, problemas y planes dentro de una arquitectura robótica. Su objetivo es facilitar la integración de planificación simbólica en sistemas desarrollados sobre ROS 2, proporcionando componentes para gestionar el dominio, el problema, la generación del plan y la ejecución de acciones.

PlanSys2 constituye la base sobre la que se plantea el proceso de replanificación dentro de la arquitectura desarrollada. El módulo desarrollado no sustituye a PlanSys2, sino que se integra como un componente complementario capaz de asistir la interpretación del contexto cuando es necesario actualizar el problema y generar un nuevo plan.

3.3.1. Componentes principales de PlanSys2

PlanSys2 organiza el proceso de planificación mediante una separación clara entre distintos componentes. Entre los elementos principales se encuentran el *Domain Expert*, el *Problem Expert*, el planificador y el ejecutor.

El *Domain Expert* se encarga de gestionar la descripción del dominio. Esta descripción incluye los tipos, predicados, funciones y acciones disponibles. Representa las reglas generales del sistema de planificación y define qué operaciones puede utilizar el planificador.

El *Problem Expert* mantiene la información correspondiente al problema concreto. Esto incluye los objetos existentes, los predicados que describen el estado actual y los objetivos que se desean alcanzar. Esta parte resulta especialmente importante, ya que la replanificación requiere modificar el problema cuando se detecta nueva información durante la ejecución.

El planificador recibe el dominio y el problema para generar una secuencia de acciones válida. Por último, el ejecutor coordina la ejecución del plan generado, conectando las acciones simbólicas con componentes ejecutables dentro del sistema ROS 2.

Esta separación permite mantener una arquitectura flexible. Cada componente cumple una función concreta y puede comunicarse con el resto mediante interfaces definidas. Esto facilita la integración del módulo desarrollado, ya que la propuesta se sitúa en torno a la actualización del problema y no requiere modificar la lógica interna del planificador.

3.3.2. Gestión del dominio y del problema

En PlanSys2, el dominio y el problema se gestionan de forma diferenciada. El dominio contiene la descripción general de las acciones disponibles, sus precondiciones y sus efectos. Esta información suele mantenerse estable durante la ejecución, ya que representa las reglas generales del sistema.

El problema, en cambio, describe la situación concreta sobre la que se desea planificar. Incluye los objetos conocidos, el estado actual y los objetivos activos. En un entorno robótico, esta información puede cambiar durante la ejecución. Un objeto puede aparecer en una ubicación diferente, una acción puede no producir el efecto esperado o una observación de percepción puede invalidar parte del estado asumido inicialmente.

Por esta razón, la gestión del problema es uno de los puntos más relevantes para la replanificación. Si el problema no se actualiza correctamente, el planificador puede generar un plan coherente desde el punto de vista formal, pero inadecuado para la situación real. Por ejemplo, si el problema indica que un objeto se encuentra en una ubicación cuando en realidad ha sido detectado en otra, el plan generado puede dirigir al robot hacia un lugar incorrecto.

El módulo propuesto se relaciona directamente con esta necesidad. Su objetivo es apoyar la

interpretación de eventos y observaciones para generar propuestas de actualización del problema. Estas propuestas no se aplican directamente, sino que deben ser validadas antes de modificar la representación mantenida por PlanSys2.

3.3.3. Generación y ejecución de planes

La generación de planes consiste en calcular una secuencia de acciones que permita alcanzar los objetivos definidos en el problema. Para ello, PlanSys2 se apoya en un planificador que recibe la descripción del dominio y del problema, y devuelve un plan si encuentra una solución válida. La Figura 3.1 ilustra este contrato: POPF recibe el dominio y el problema en PDDL y devuelve, si existe, una secuencia de acciones.

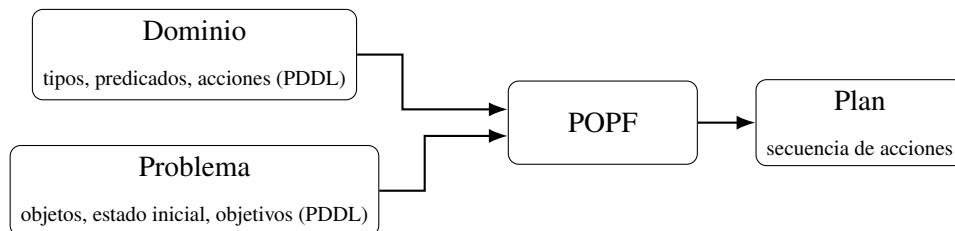


Figura 3.1: Contrato de entrada y salida de POPF.

El plan generado contiene acciones ordenadas que deben ejecutarse para transformar el estado inicial en un estado que satisfaga el objetivo. Cada acción del plan corresponde a una acción definida en el dominio y debe cumplir sus precondiciones en el momento de ser ejecutada.

Una vez generado el plan, PlanSys2 coordina su ejecución. Las acciones simbólicas se asocian con componentes ejecutables dentro del sistema ROS 2, normalmente implementados como nodos o servidores de acción. Estos componentes pueden representar comportamientos del robot, como desplazarse a una ubicación, manipular un objeto, realizar una comprobación o interactuar con el entorno.

La ejecución del plan no debe entenderse como una reproducción cerrada de la secuencia generada. Durante la ejecución pueden producirse fallos, retrasos o resultados inesperados. Por ello, el sistema necesita monitorizar el progreso de las acciones y detectar si el plan sigue siendo válido.

Cuando una acción falla o el estado del mundo cambia, puede ser necesario detener la ejecución del plan actual y generar uno nuevo. Este proceso constituye el punto de partida de la propuesta planteada.

3.3.4. POPF como planificador externo

PlanSys2 puede apoyarse en planificadores externos para generar planes. En el sistema desarrollado, POPF [3] se considera como el planificador encargado de calcular una secuencia de acciones válida a partir de la descripción formal del dominio y del problema. POPF es un planificador *satisficing* basado en *forward-chaining partial-order planning*, distribuido como ejecutable estándar en la integración por defecto de PlanSys2.

El papel de POPF se limita a la generación del plan. Recibe una representación formal del problema de planificación y devuelve una secuencia de acciones si encuentra una solución. No ejecuta acciones en el robot, no interpreta información sensorial sin procesar y no razona directamente sobre logs, eventos textuales u observaciones ambiguas.

Esta separación es importante porque muestra una de las limitaciones que motivan el uso de un módulo auxiliar. El planificador solo puede razonar con la información que recibe en forma simbólica. Si el estado del problema está incompleto, desactualizado o no refleja correctamente la situación real, el plan generado puede no ser adecuado.

Por tanto, el interés de la propuesta no se encuentra en modificar el funcionamiento interno de POPF, sino en mejorar la información que se proporciona al proceso de planificación cuando el sistema necesita replanificar. El modelo de lenguaje se introduce antes de la llamada al planificador, como apoyo para interpretar contexto y proponer actualizaciones del problema.

3.3.5. Limitaciones ante información incompleta

Una limitación importante de los planificadores simbólicos es su dependencia de una descripción explícita del estado. Si un hecho no está representado, si una observación no ha sido incorporada al problema o si el estado contiene información incorrecta, el planificador no puede resolver por sí solo esa discrepancia.

En robótica, esta situación aparece con frecuencia. Los sensores pueden detectar información nueva, las acciones pueden fallar, los objetos pueden cambiar de ubicación y el estado interno puede dejar de coincidir con la realidad. En estos casos, el problema no consiste únicamente en generar un nuevo plan, sino en decidir qué información debe actualizarse antes de volver a planificar.

Esta dificultad conecta directamente con el objetivo de la propuesta. El módulo desarrollado utiliza modelos de lenguaje como apoyo para interpretar eventos y contexto, pero mantiene la necesidad de validar la información antes de incorporarla al proceso de replanificación. De esta forma, PlanSys2 conserva el papel de sistema formal de planificación, mientras que el LLM actúa como una herramienta auxiliar dentro de una arquitectura controlada.

3.4. Replanificación en sistemas robóticos

La replanificación aparece cuando el plan inicial deja de ser válido, conveniente o ejecutable debido a cambios durante la ejecución. En un sistema robótico, esto puede ocurrir por múltiples causas: fallos de acciones, cambios en el entorno, nueva información procedente de sensores o discrepancias entre el estado esperado y el estado observado.

A diferencia de la planificación inicial, la replanificación parte de un sistema que ya se encuentra en ejecución. Esto significa que el robot ha realizado parte de la tarea, el estado ha cambiado y pueden existir efectos parciales producidos por acciones anteriores. Por tanto, no basta con volver al problema original, sino que es necesario actualizar la representación del estado antes de generar un nuevo plan.

3.4.1. Necesidad de replanificar

La necesidad de replanificar surge porque los planes se generan a partir de una representación concreta del mundo, pero dicha representación puede dejar de ser válida. Un plan que era correcto en el momento inicial puede no serlo después de que cambie el entorno, falle una acción o aparezca información nueva.

Por ejemplo, si un robot debe recoger un objeto en una ubicación determinada y durante la ejecución descubre que el objeto se encuentra en otro lugar, el plan inicial deja de ser adecuado. En ese caso, el sistema debe actualizar el estado y generar un nuevo plan que tenga en cuenta la nueva ubicación.

La replanificación permite que el sistema mantenga cierta autonomía ante situaciones no previstas. Sin ella, cualquier desviación respecto al estado esperado podría requerir intervención manual o provocar el fallo completo de la tarea.

En la arquitectura planteada, la replanificación no se entiende únicamente como una nueva llamada al planificador. Antes de solicitar un nuevo plan, el sistema debe interpretar qué ha ocurrido, qué información es relevante y cómo debe modificarse el problema de planificación. Esta fase previa es precisamente donde se introduce el uso de modelos de lenguaje.

3.4.2. Fallos de ejecución y cambios en el entorno

Los fallos de ejecución son una causa habitual de replanificación. Una acción puede no completarse correctamente porque el robot no alcanza la posición esperada, porque un objeto no está disponible, porque una condición necesaria deja de cumplirse o porque la percepción no confirma el resultado esperado.

Los cambios en el entorno también pueden invalidar un plan. En entornos dinámicos, otros

agentes pueden mover objetos, bloquear zonas o modificar condiciones relevantes para el robot. Además, la percepción del robot puede descubrir información que no estaba disponible en el momento de generar el plan inicial.

Estos casos muestran que la planificación robótica no puede entenderse como un proceso completamente cerrado. El sistema debe mantener una relación continua entre planificación, ejecución y actualización del estado. La ejecución proporciona información nueva, y dicha información puede obligar a modificar el problema antes de volver a planificar.

3.4.3. Actualización del problema de planificación

Antes de generar un nuevo plan, el sistema debe actualizar el problema de planificación. Esta actualización consiste en incorporar información nueva y eliminar o modificar aquellos hechos que ya no son válidos.

La actualización del problema puede parecer sencilla, pero en la práctica implica varios retos. El sistema debe decidir qué eventos son relevantes, qué observaciones son fiables y cómo se traducen al lenguaje simbólico del planificador. Además, debe evitar introducir contradicciones que hagan imposible generar un nuevo plan.

Por ejemplo, si una observación indica que un objeto ha sido detectado en una nueva ubicación, puede ser necesario eliminar el predicado que lo situaba en la ubicación anterior y añadir un nuevo predicado con la ubicación actual. Sin embargo, esta actualización solo debería realizarse si la observación es relevante, está suficientemente respaldada y es compatible con los objetos y predicados definidos en el dominio.

En el sistema desarrollado, los modelos de lenguaje se utilizan como apoyo para interpretar información contextual y generar propuestas de actualización. No obstante, dichas propuestas no se aplican directamente sobre PlanSys2. Primero deben procesarse y validarse para comprobar que son compatibles con el problema de planificación y con el estado disponible.

3.4.4. Dificultades de la replanificación en entornos abiertos

La robótica real introduce incertidumbre. Los sensores pueden tener errores, los actuadores pueden no comportarse exactamente como se espera y el entorno puede cambiar de forma impredecible. Esta incertidumbre dificulta la actualización del estado y la generación de nuevos planes.

Otra dificultad es la interpretación de eventos. No todos los eventos tienen la misma importancia para la planificación. Algunos pueden ser meramente informativos, mientras que otros pueden invalidar completamente el plan actual. Distinguir entre ambos casos requiere contexto.

Además, la información relevante para replanificar puede aparecer distribuida entre distintos componentes del sistema. Una acción puede fallar, un nodo de percepción puede detectar un objeto en una ubicación inesperada y el estado simbólico puede seguir conteniendo información anterior. El sistema necesita relacionar estos elementos para decidir qué actualización del problema tiene sentido.

3.5. Uso de LLMs como apoyo a la replanificación simbólica

Los modelos de lenguaje (LLMs) son sistemas capaces de procesar y generar texto a partir de un contexto de entrada. Aunque suelen asociarse a modelos de gran tamaño ejecutados en infraestructura remota, en este trabajo el interés se centra en modelos pequeños ejecutados localmente, como la familia Qwen2.5 [4]: su menor capacidad de razonamiento general se compensa restringiendo la tarea mediante un *prompt* estructurado y reglas explícitas, lo que los hace adecuados para un papel auxiliar y verificable dentro del sistema de planificación.

En planificación robótica, esta capacidad puede utilizarse para interpretar información procedente de logs, eventos de ejecución, observaciones del entorno o mensajes generados por otros componentes del sistema. Sin embargo, en la arquitectura propuesta los LLMs no se utilizan como sustitutos del planificador simbólico. Su papel es auxiliar: ayudan a interpretar información contextual y a generar propuestas que posteriormente deben validarse antes de modificar el problema de planificación.

La Figura 3.2 resume el ciclo conceptual que las siguientes subsecciones detallan paso a paso. El bloque resaltado indica el punto donde se inserta el modelo de lenguaje; el resto del ciclo conserva la estructura clásica de planificación, ejecución y replanificación, con las garantías formales delegadas a PlanSys2 y POPF.

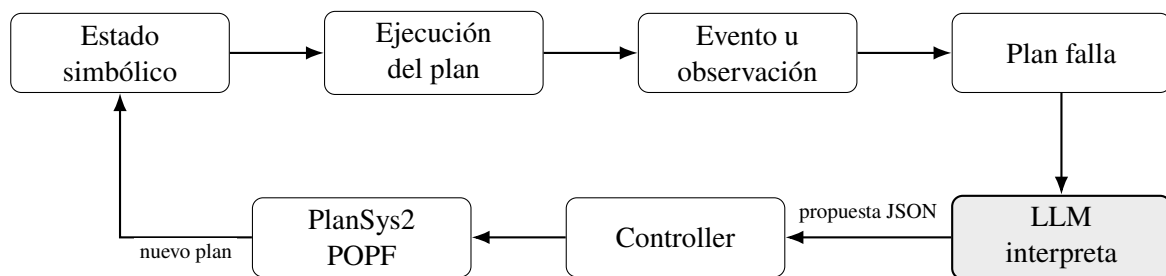


Figura 3.2: Ciclo de replanificación asistida por LLM.

3.5.1. Motivación para introducir LLMs

La motivación principal para introducir modelos de lenguaje no es sustituir al planificador simbólico, sino complementar sus limitaciones ante información contextual. Los planificado-

res clásicos trabajan sobre representaciones formales, normalmente compuestas por objetos, predicados, acciones y objetivos. Esta representación permite generar planes coherentes desde el punto de vista lógico, pero requiere que la información relevante del entorno haya sido transformada previamente a un formato simbólico.

En un sistema robótico en ejecución, gran parte de la información disponible no aparece inicialmente en esa forma. Los fallos de acciones, las observaciones de percepción, los mensajes de estado o las trazas de ejecución suelen expresarse como eventos textuales, logs o datos intermedios producidos por distintos componentes del sistema. Esta información puede ser relevante para decidir cómo actualizar el problema de planificación, pero no puede ser utilizada directamente por un planificador como POPF.

En este contexto, los LLMs se utilizan como una herramienta auxiliar para interpretar información contextual y generar propuestas de actualización. Su función consiste en analizar los eventos disponibles, identificar qué elementos pueden afectar al plan actual y sugerir posibles modificaciones del estado simbólico.

La elección de modelos pequeños y locales no es accesorio, sino parte de la propuesta. Evita la dependencia de servicios externos, acota el coste computacional y permite orientar el sistema a hardware con recursos limitados. La contrapartida, una menor robustez del modelo frente a modelos de mayor tamaño, se gestiona mediante el diseño del *prompt* y la validación posterior, y su impacto se analiza experimentalmente en el Capítulo 6.

Sin embargo, estas propuestas no se consideran automáticamente válidas. El sistema debe procesarlas, comprobar su formato y validar su coherencia antes de modificar el problema gestionado por PlanSys2. Esta separación permite aprovechar la flexibilidad interpretativa de los modelos de lenguaje sin delegar en ellos la responsabilidad final de la planificación.

3.5.2. Construcción del contexto de entrada

El contexto proporcionado al modelo de lenguaje constituye uno de los elementos más importantes del sistema. Este contexto debe contener la información necesaria para interpretar la situación de ejecución, pero evitando incluir datos irrelevantes que puedan dificultar la respuesta del modelo o introducir ruido en el proceso.

En el sistema desarrollado, el contexto puede incluir información sobre el estado actual del problema de planificación, el objetivo activo, las acciones ejecutadas, el resultado de dichas acciones y las observaciones relevantes obtenidas durante la ejecución. También puede incluir eventos de percepción que indiquen cambios en el entorno, como la detección de un objeto en una ubicación distinta a la esperada.

Dentro de ese contexto, la observación en lenguaje natural cumple una función adicional a la de describir un evento: actúa como superficie de especialización del modelo. A través de

ella, la aplicación integradora introduce información y restricciones propias de su caso de uso, condicionando el comportamiento de un modelo de propósito general sin necesidad de reentrenarlo. La especialización se traslada así del modelo al contexto.

La construcción del contexto no consiste simplemente en enviar todos los logs disponibles al modelo. Es necesario seleccionar, ordenar y estructurar la información para que el LLM pueda razonar sobre ella de forma útil. Un contexto excesivamente largo o desordenado puede provocar respuestas ambiguas, mientras que un contexto demasiado reducido puede omitir información necesaria para interpretar correctamente la situación.

Por ello, el módulo desarrollado actúa como una capa intermedia entre el sistema robótico y el modelo de lenguaje. Su función es recopilar los eventos relevantes, construir una representación textual controlada y solicitar al modelo una respuesta en un formato que pueda ser procesado posteriormente.

Esta fase es especialmente importante porque condiciona directamente la calidad de la respuesta generada. Si el contexto no incluye la información necesaria para justificar una modificación del problema, el modelo puede producir una propuesta incompleta o basada en supuestos no respaldados. Del mismo modo, si el contexto contiene información irrelevante o contradictoria, el modelo puede centrar su respuesta en aspectos que no afectan realmente a la planificación.

3.5.3. Interpretación de eventos, observaciones y trazas

Durante la ejecución de un plan, el sistema puede generar distintos tipos de información. Las acciones pueden informar de su inicio, progreso, finalización o fallo. Los nodos de percepción pueden generar observaciones sobre objetos, ubicaciones o cambios en el entorno. Otros componentes pueden producir mensajes de estado o advertencias relevantes para la tarea.

No toda esta información tiene el mismo valor para la replanificación. Algunos eventos pueden ser secundarios, mientras que otros pueden indicar que el plan actual ha dejado de ser válido. Por ejemplo, que una acción tarde más de lo esperado puede ser un evento relevante para la monitorización, pero no necesariamente obliga a cambiar el problema de planificación. En cambio, detectar que el objeto objetivo no está en la ubicación esperada puede requerir una actualización del estado y una nueva planificación.

Los LLMs pueden ayudar en esta fase porque son capaces de procesar descripciones textuales de eventos y relacionarlas con una situación global. A partir de un conjunto de observaciones, el modelo puede identificar qué hecho parece haber cambiado, qué acción se ha visto afectada o qué información debería considerarse para actualizar el problema.

Este uso resulta especialmente útil cuando la información relevante no aparece como un único evento aislado, sino como una combinación de varios mensajes. Por ejemplo, una acción puede fallar al buscar un objeto, y posteriormente un módulo de percepción puede detectar ese mismo

objeto en otra ubicación. La interpretación conjunta de ambos eventos permite inferir que el problema de planificación debe actualizar la localización del objeto antes de generar un nuevo plan.

3.5.4. Generación de propuestas de actualización

Una vez construido el contexto, el modelo de lenguaje puede generar una propuesta de actualización. Esta propuesta puede indicar qué hechos deberían añadirse, eliminarse o modificarse en el problema de planificación.

Por ejemplo, si el sistema esperaba encontrar un objeto en una ubicación concreta, pero la percepción lo detecta en otra, el modelo podría proponer eliminar el predicado que situaba el objeto en la ubicación anterior y añadir un nuevo predicado con la ubicación actual. De esta forma, el problema de planificación quedaría alineado con la nueva información disponible.

Sin embargo, este proceso debe entenderse como una generación de hipótesis, no como una modificación directa del estado. El modelo puede equivocarse, interpretar incorrectamente el contexto o proponer cambios incompatibles con el dominio. Por ello, su salida debe considerarse una sugerencia que necesita ser comprobada.

El interés de utilizar un LLM en esta fase se encuentra en su capacidad para transformar información contextual en una propuesta más cercana al lenguaje simbólico del planificador. El modelo actúa como un puente entre eventos textuales y posibles modificaciones formales del problema. Aun así, la responsabilidad de aceptar o rechazar esa propuesta permanece en el sistema.

3.5.5. Formato estructurado de salida

Para que la respuesta de un LLM sea útil dentro de un sistema de planificación, no basta con obtener una explicación en lenguaje natural. El sistema necesita una salida que pueda interpretarse de forma automática y que pueda relacionarse con los elementos del dominio de planificación.

Por este motivo, el modelo debe generar respuestas estructuradas. Estas respuestas pueden indicar, por ejemplo, qué hecho del estado debe añadirse, qué predicado debe eliminarse, qué objeto ha cambiado de ubicación o qué condición ha invalidado el plan actual. El objetivo es reducir la ambigüedad de la salida y facilitar su posterior validación.

El uso de un formato estructurado también permite separar la interpretación del modelo de la aplicación efectiva de los cambios. El LLM puede proponer una modificación, pero el sistema mantiene el control sobre si dicha modificación es aceptada o descartada. Esta separación resulta esencial para evitar que una respuesta incorrecta del modelo afecte directamente al estado de

planificación.

Además, una salida estructurada facilita la comparación entre distintos modelos durante la experimentación. Si todos los modelos deben responder siguiendo un formato común, resulta más sencillo analizar qué propuestas son correctas, cuáles son incompletas y cuáles introducen errores. Esto permite evaluar no solo la calidad lingüística de la respuesta, sino su utilidad real dentro del proceso de replanificación.

3.5.6. Validación antes de modificar el problema

La validación de las respuestas del modelo es una parte fundamental de la arquitectura propuesta. Aunque un LLM pueda generar una respuesta aparentemente coherente, no existe garantía de que dicha respuesta sea correcta, completa o compatible con el dominio de planificación.

Antes de modificar el problema gestionado por PlanSys2, el sistema debe comprobar que la propuesta generada por el modelo cumple una serie de condiciones. En primer lugar, debe respetar el formato esperado. En segundo lugar, los objetos, predicados y acciones mencionados deben existir dentro del dominio o del problema actual. En tercer lugar, la propuesta no debe introducir contradicciones evidentes con el estado disponible ni con observaciones recientes.

Esta validación permite mantener al LLM en un papel auxiliar. El modelo ayuda a interpretar contexto y a generar hipótesis de actualización, pero no modifica directamente el estado simbólico ni sustituye al planificador. De esta forma, el sistema combina la flexibilidad interpretativa de los modelos de lenguaje con el control formal propio de la planificación simbólica.

La validación también permite detectar respuestas incompletas. Por ejemplo, si el modelo propone añadir que un objeto se encuentra en una nueva ubicación, pero no propone eliminar la ubicación anterior, el sistema puede detectar una posible inconsistencia. Del mismo modo, si el modelo utiliza un predicado que no existe en el dominio, la propuesta debe descartarse o corregirse antes de afectar al problema de planificación.

3.5.7. Relación entre LLM, PlanSys2 y POPF

La arquitectura planteada mantiene una separación clara entre las responsabilidades del modelo de lenguaje, PlanSys2 y el planificador externo. El LLM se encarga de interpretar información contextual y generar propuestas de actualización. PlanSys2 mantiene la representación formal del dominio y del problema, además de coordinar la generación y ejecución de planes. POPF, por su parte, actúa como planificador simbólico encargado de calcular una secuencia de acciones válida a partir del dominio y del problema actualizados.

Esta separación evita delegar la planificación completa en el modelo de lenguaje. El LLM no genera directamente el plan final ni decide qué acciones debe ejecutar el robot. Su papel se limita

a apoyar la fase previa a la replanificación, ayudando a transformar información contextual en posibles modificaciones del problema.

De este modo, el proceso general puede entenderse como un ciclo. Durante la ejecución, el sistema recopila eventos y observaciones. Cuando se detecta una situación que puede invalidar el plan, el módulo construye un contexto para el LLM. El modelo genera una propuesta de actualización, dicha propuesta se valida y, si es aceptada, se modifica el problema de planificación. Finalmente, PlanSys2 solicita a POPF un nuevo plan compatible con el estado actualizado.

Esta organización permite conservar las ventajas de la planificación simbólica, como la representación formal de acciones, precondiciones y efectos, al mismo tiempo que se incorpora una capa flexible para interpretar información contextual. El resultado es una arquitectura en la que el LLM no sustituye al razonamiento formal, sino que lo complementa en aquellos puntos donde la información del entorno debe ser interpretada antes de poder ser utilizada por el planificador.

3.6. Estado del arte

El uso de modelos de lenguaje como apoyo a la planificación robótica ha sido abordado desde varios enfoques, en un contexto más amplio de planificación formal basada en lenguajes como PDDL [5] y de evaluación comparativa mediante competiciones y bancos de prueba de planificación [6]. Un primer grupo de trabajos emplea el LLM como traductor de lenguaje natural a representaciones formales, delegando la búsqueda en un planificador clásico, como ocurre en LLM+P [7]. Un segundo grupo lo utiliza para seleccionar acciones afianzadas en las capacidades del robot, combinando el modelo con estimadores de viabilidad, como en SayCan [8]. Un tercero hace que el modelo genere directamente programas o secuencias de actuación a partir de la instrucción, como en ProgPrompt y Text2Motion [9], [10]. En la mayoría de estos casos el modelo participa en el lazo de generación del plan y se ejecuta sobre modelos grandes o servicios remotos.

Sobre este panorama, el presente trabajo se posiciona en dos aspectos. Primero, el LLM no genera el plan ni ejecuta acciones, sino que produce una propuesta estructurada de actualización del problema que un planificador simbólico, POPF [3], resuelve después, conservando las garantías formales de la planificación clásica. Segundo, frente al uso habitual de modelos grandes o remotos, aquí se emplea un modelo pequeño ejecutado localmente, especializado por contexto, no por reentrenamiento, y sometido a validación estructural antes de influir en el estado. El sistema se construye sobre PlanSys2 [2] y ROS 2 [1], con inferencia local mediante llama.cpp [11] a través de llama_ros. llama_ros es un *wrapper* de llama.cpp para ROS 2 que expone la inferencia local como acciones, servicios y *topics* estándar del *middleware* [12]; la viabilidad práctica de este despliegue se apoya además en técnicas de cuantización y ejecución eficiente de modelos [13], [14].

La Tabla 3.2 resume las diferencias entre los enfoques discutidos y la propuesta de este trabajo.

Trabajo	Papel del LLM	Ejecución	Salida del LLM	Validación posterior
LLM+P [7]	Traducción de lenguaje natural a PDDL	Remota, modelo grande	Problema u objetivo en PDDL	Implícita: el planificador rechaza problemas no resolubles
SayCan [8]	Selección de acción del catálogo del robot	Remota, modelo grande	Acción afianzada en capacidades	Estimador de <i>affordance</i> del robot
ProgPrompt / Text2-Motion [9], [10]	Generación directa de programa o secuencia de actuación	Remota, modelo grande	Código o secuencia ejecutable	Limitada o ausente
Esta propuesta	Propuesta de actualización del problema de planificación	Local, modelo pequeño (Qwen2.5-3B)	JSON estructurado	Estructural en el módulo; semántica delegada a PlanSys2 y POPF

Tabla 3.2: Comparativa del papel del LLM en enfoques de planificación robótica asistida por modelos de lenguaje. La fila destacada corresponde a la propuesta de este trabajo.

3.7. Limitaciones y riesgos del uso de LLMs

El uso de modelos de lenguaje introduce riesgos específicos que deben tenerse en cuenta, especialmente cuando se aplican en sistemas robóticos; trabajos recientes señalan limitaciones de robustez, alucinación y razonamiento planificador en LLMs [15], [16], [17]. A diferencia de un planificador formal, un LLM no garantiza por sí mismo que sus respuestas sean correctas, completas o coherentes con el estado real del sistema.

3.7.1. Alucinaciones e información no verificable

Uno de los riesgos más conocidos de los modelos de lenguaje es la generación de información inventada o no verificable. El modelo puede producir una respuesta aparentemente coherente, pero basada en supuestos que no aparecen en el contexto proporcionado.

En robótica, este comportamiento puede ser problemático. Si el modelo inventa la existencia de un objeto, una ubicación o una condición del entorno, y esa información se incorpora al sistema de planificación, el nuevo plan podría ser incorrecto.

Por este motivo, el sistema debe evitar aceptar directamente información no respaldada por el estado, los eventos o las observaciones disponibles. La trazabilidad de la información es esencial para determinar de dónde procede cada propuesta generada. Una modificación del problema solo debería aceptarse si puede relacionarse con información disponible en el contexto de entrada o con reglas explícitas del sistema.

3.7.2. Inconsistencias con el estado real del sistema

Otro riesgo es que la respuesta del modelo sea inconsistente con el estado real del sistema. Por ejemplo, el modelo podría proponer una modificación que contradice una observación reciente, una condición necesaria del dominio o un hecho ya establecido en el problema.

Estas inconsistencias pueden aparecer porque el modelo no mantiene una representación formal interna del sistema comparable a la de un planificador. Aunque pueda interpretar texto de forma flexible, no garantiza automáticamente la consistencia lógica de sus respuestas.

Por ello, el módulo desarrollado debe contemplar mecanismos para detectar respuestas inválidas, incompletas o incompatibles con el contexto disponible. Esta validación es necesaria antes de utilizar cualquier salida del modelo en el proceso de replanificación.

3.7.3. Dependencia del prompt

Las respuestas de un LLM dependen en gran medida del prompt utilizado. La forma en que se presenta el contexto, las instrucciones dadas al modelo y el formato de salida solicitado pueden influir notablemente en la calidad de la respuesta.

Esta dependencia implica que el diseño del prompt es una parte relevante del sistema. Un prompt demasiado ambiguo puede generar respuestas poco útiles, mientras que un prompt demasiado restrictivo puede limitar la capacidad del modelo para interpretar situaciones complejas.

La generación de prompts se considera parte del diseño del módulo. El objetivo es proporcionar al modelo información suficiente para interpretar el contexto, pero imponiendo restricciones claras sobre el tipo de respuesta esperada. En particular, el prompt debe indicar que el modelo no debe inventar información, que debe basarse únicamente en el contexto proporcionado y que debe responder siguiendo el formato establecido.

3.7.4. Variabilidad entre ejecuciones

Los modelos de lenguaje pueden producir respuestas diferentes ante entradas similares, especialmente si se utilizan configuraciones que permiten cierta aleatoriedad. Esta variabilidad puede

ser útil en tareas creativas, pero resulta problemática en sistemas que requieren reproducibilidad y control.

En planificación robótica, la variabilidad debe tratarse con cuidado. Si dos ejecuciones similares producen propuestas distintas, el sistema debe ser capaz de evaluar cuál de ellas es válida o si ambas deben descartarse.

Por este motivo, la experimentación debe considerar no solo si el modelo produce una respuesta aparentemente correcta, sino también la estabilidad, consistencia y utilidad de sus respuestas en distintos escenarios. Un modelo puede ser útil si genera propuestas correctas de forma frecuente, pero también debe analizarse si sus errores son detectables mediante los mecanismos de validación definidos.

3.7.5. Necesidad de validación externa

Debido a los riesgos anteriores, las respuestas generadas por un LLM deben validarse antes de incorporarse al proceso de planificación. Esta validación puede incluir comprobaciones de formato, consistencia con el estado, compatibilidad con el dominio y ausencia de contradicciones evidentes.

La validación externa permite separar la capacidad interpretativa del modelo de la responsabilidad final del sistema. El modelo puede proponer, pero el sistema debe decidir si la propuesta es aceptable.

Esta separación constituye una de las ideas principales de la arquitectura propuesta. El objetivo no es delegar la planificación en un LLM, sino utilizarlo como una herramienta auxiliar dentro de una arquitectura controlada y verificable. De esta forma, el sistema puede beneficiarse de la capacidad de los modelos de lenguaje para interpretar contexto, sin renunciar al control formal necesario en un sistema robótico basado en planificación simbólica.

A modo de cierre, la Tabla 3.3 resume los cinco riesgos discutidos en las subsecciones anteriores y la mitigación que recibe cada uno, ya sea por el propio módulo o por delegación a PlanSys2 y POPF.

Riesgo	Cómo se manifiesta	Mitigación
Alucinación	El modelo introduce objetos, predicados o ubicaciones que no aparecen en el contexto.	Aplicación delegada: PlanSys2 rechaza al aplicarlos los predicados que no encajan con la firma del dominio.
Inconsistencia con el estado	La propuesta contradice observaciones recientes o predicados ya establecidos en el problema.	Validación delegada al planificador: POPF no devuelve plan si el problema resultante es insatisfacible.
Dependencia del prompt	La calidad varía con la formulación, el orden y el nivel de detalle del contexto.	Plantilla fija con orden estático → dinámico y formato de salida requerido.
Variabilidad entre ejecuciones	La misma entrada puede producir respuestas distintas.	<code>temperature = 0</code> y reutilización de la KV-cache mediante el orden estático → dinámico del prompt.
Salida no procesable	JSON malformado, campos ausentes o tipos incorrectos.	Extracción tolerante del bloque JSON y clasificación ERROR si el parseo falla.

Tabla 3.3: Resumen de los riesgos del uso de LLMs en el módulo y mitigaciones aplicadas o delegadas.

3.8. Síntesis del capítulo

En este capítulo se han presentado los fundamentos tecnológicos sobre los que se apoya el sistema desarrollado. ROS 2 proporciona la infraestructura de comunicación e integración robótica, permitiendo organizar el sistema mediante nodos y mecanismos de comunicación distribuidos. PlanSys2 ofrece el marco de planificación simbólica, gestionando el dominio, el problema, la generación de planes y la ejecución de acciones. POPF actúa como planificador externo encargado de generar planes a partir de una representación formal del dominio y del problema.

A partir de estas bases, se ha analizado la necesidad de replanificación en sistemas robóticos. Los planes se generan sobre una representación concreta del mundo, pero dicha representación puede dejar de ser válida durante la ejecución debido a fallos, cambios en el entorno u observaciones nuevas. En estos casos, el problema de planificación debe actualizarse antes de solicitar un nuevo plan.

Finalmente, se ha introducido el papel de los modelos de lenguaje dentro de este proceso. Los LLMs se plantean como una herramienta auxiliar para interpretar información contextual,

resumir eventos y generar propuestas de actualización del problema. Sin embargo, debido a sus limitaciones, estas propuestas deben validarse antes de influir en el sistema de planificación.

Esta separación de responsabilidades define el enfoque general de la propuesta: combinar la robustez formal de la planificación simbólica con la flexibilidad interpretativa de los modelos de lenguaje, manteniendo siempre mecanismos de control y validación sobre la información utilizada para replanificar.

Capítulo 4

Diseño del módulo propuesto

4.1. Visión general del sistema

El sistema propuesto se diseña como un módulo complementario capaz de integrarse con PlanSys2 para asistir procesos de replanificación mediante modelos de lenguaje. La idea principal es mantener el planificador formal como núcleo responsable de generar planes válidos, mientras que el LLM se utiliza para interpretar contexto y proponer modificaciones a partir de eventos o información recibida durante la ejecución.

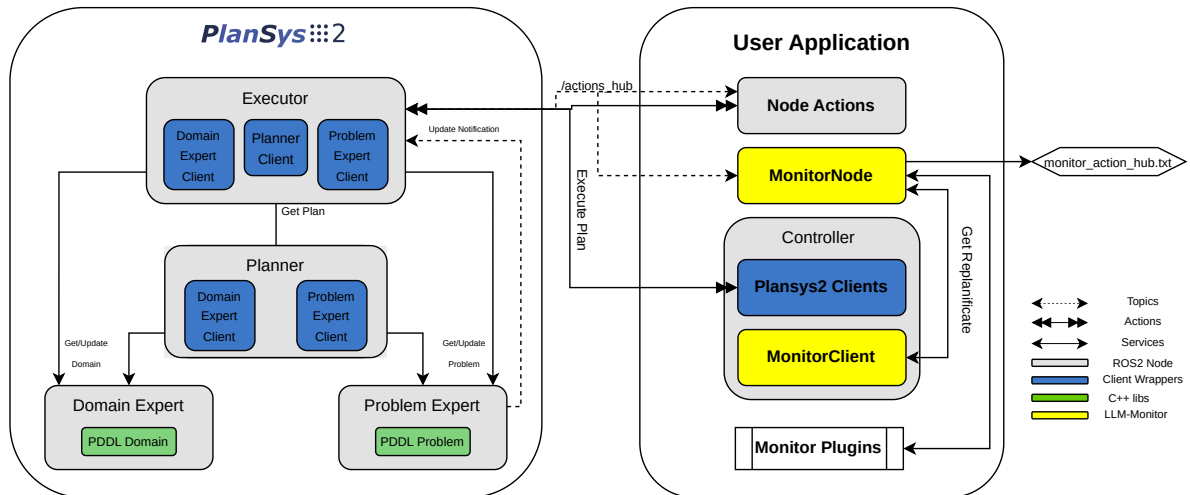


Figura 4.1: Visión general del sistema

La Figura 4.1 distingue tres planos. El núcleo de PlanSys2 (*Domain Expert*, *Problem Expert*, *Planner* con POPF y *Executor*) permanece intacto. El contenedor resaltado agrupa la contribución de este trabajo: el *MonitorNode* y el *plugin* de inferencia local. El tercer plano es la aplicación integradora, que conserva la decisión final sobre si aplicar una propuesta. La leyenda del diagrama identifica los *topics*, acciones y servicios empleados, y diferencia visualmente los nodos ROS 2 de los componentes implementados en C++.

4.2. Arquitectura del módulo

La arquitectura del módulo se organiza en componentes con responsabilidades diferenciadas. Esta separación permite mejorar la mantenibilidad, facilitar las pruebas y aislar la dependencia con los modelos de lenguaje. Además, el diseño modular permite separar el flujo de replanificación en etapas independientes: captura de contexto, orquestación, construcción del *prompt*, inferencia, procesamiento de la respuesta e integración con el sistema de planificación.

La descomposición en bloques B1–B6 materializa el objetivo OE2, al separar captura de contexto, orquestación, construcción del *prompt*, inferencia, procesamiento y cliente de integración. Esta división evita acoplar directamente la lógica de PlanSys2 con un modelo de lenguaje concreto. De esta forma, el backend LLM puede sustituirse o ampliarse sin modificar el resto de la arquitectura, y las respuestas generadas pueden procesarse y validarse antes de influir sobre el problema de planificación.

- **B1** – Entrada de contexto y eventos.
- **B2** – Orquestación y ciclo de vida.
- **B3** – Construcción de prompts.
- **B4** – Inferencia LLM.
- **B5** – Parsing y validación de respuestas.
- **B6** – Cliente e integración con PlanSys2.

Esta enumeración no implica que los seis bloques residan en el mismo nodo ROS 2. Los bloques B1–B5 se implementan dentro del nodo *MonitorNode*, perteneciente al paquete *plansys2_monitor*, y constituyen el módulo propiamente dicho: captura de contexto, orquestación, construcción del *prompt*, inferencia LLM y parsing de respuestas. El bloque B6, en cambio, vive fuera del módulo: corresponde al cliente C++ (*MonitorClient*) que se enlaza en la aplicación integradora y consume el servicio *monitor/get_proposal*. Esta separación es deliberada y materializa el principio de que el módulo expone una interfaz reutilizable, mientras

la política de aceptación e integración con `problem_expert` queda fuera de su alcance, en manos de quien lo integra.

La Tabla 5.1 resume el uso principal y la salida asociada a cada bloque funcional.

4.3. Integración con PlanSys2

La integración con PlanSys2 se plantea de forma desacoplada. El módulo no pretende sustituir los componentes principales del planificador, sino actuar como una capa auxiliar que recibe información de ejecución, genera propuestas estructuradas y las devuelve a un nodo integrador externo.

Una decisión central del diseño es no modificar el código interno de PlanSys2 ni depender directamente de sus componentes internos de planificación, como el `problem_expert`. El módulo consume eventos compatibles con el `topic /actions_hub`, construye contexto a partir de ellos y expone una interfaz de consulta para que otros nodos soliciten asistencia. La aplicación integradora conserva la responsabilidad de validar la propuesta recibida, decidir si debe aplicarse y, en caso afirmativo, solicitar una nueva planificación.

Esta decisión aborda el requisito RNF-01, ya que el núcleo de PlanSys2 permanece intacto y la interacción se realiza mediante interfaces externas. Además, permite que el módulo sea reutilizable y que pueda evolucionar sin depender de cambios profundos en el núcleo de PlanSys2. Además, facilita su adaptación a otros planificadores siempre que proporcionen eventos de ejecución equivalentes o una capa de traducción compatible.

4.4. Flujo de información del sistema

El flujo de información comienza con la recepción de eventos o datos contextuales mediante el `topic /actions_hub`. Este `topic` contiene la información del flujo de acciones de PlanSys2, y cada acción creada por el usuario puede incorporar realimentación a sus mensajes. A partir de esta información, el módulo construye una representación textual estructurada que se envía al modelo de lenguaje. La respuesta se procesa, se comprueba y se transforma en una salida utilizable por el sistema de planificación.

Aunque `/actions_hub` se ha mencionado como punto de entrada del flujo, conviene precisar qué información transporta, ya que de ello depende qué puede razonar el modelo en la fase posterior. PlanSys2 publica en este `topic` las transiciones del ciclo de vida de cada acción del plan: cada mensaje incluye el identificador y los argumentos de la acción, su estado actual y, opcionalmente, un campo de realimentación textual redactado por la propia acción. Los tipos de evento relevantes para la replanificación son REQUEST (PlanSys2 solicita una acción al ejecutor), RESPONSE (el

ejecutor acepta o rechaza la asignación), FEEDBACK (progreso parcial durante la ejecución) y los estados terminales FINISH y CANCEL, que indican éxito o fallo. El módulo se suscribe a este *topic* y mantiene una traza histórica que después se resume mediante `summarizeActionLog()` (Sección 5.4.1) para inyectarla en el *prompt* como contexto de ejecución. La elección del modo *limited*, que conserva sólo las transiciones terminales, responde precisamente a que el cuerpo de FEEDBACK puede contener varias decenas de detecciones de percepción que no aportan información para la replanificación pero sí inflarían el contexto.

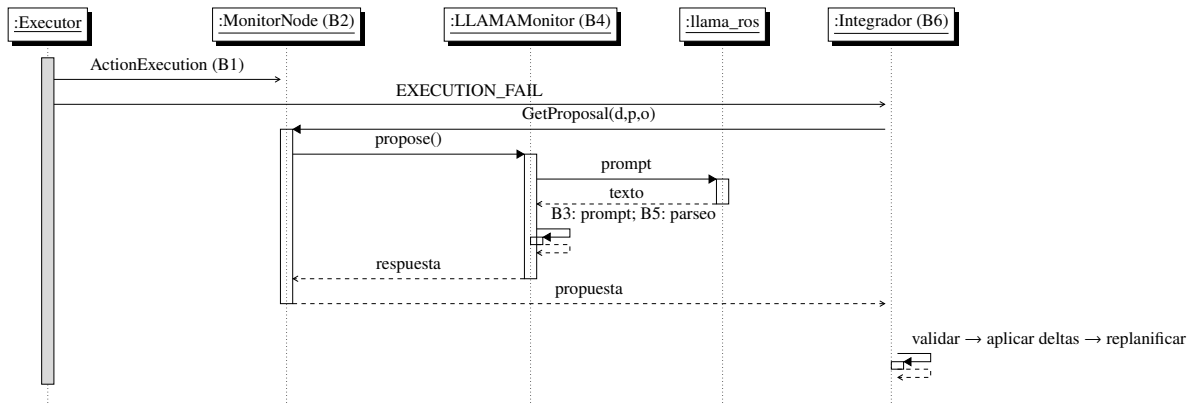


Figura 4.2: Flujo de replanificación

La Figura 4.2 muestra el diagrama de secuencia del flujo completo, desde la detección del fallo de ejecución hasta la consulta al monitor y la replanificación posterior.

4.4.1. Flujo del nodo integrador

El módulo expone una interfaz; la política de cuándo replanificar y qué propuestas aplicar recae en un nodo integrador externo, propio de cada aplicación. El demostrador de este trabajo implementa ese nodo como referencia de integración. Su ciclo es: (1) detectar, durante la ejecución del plan, que una acción ha fallado o que una observación de percepción contradice el estado simbólico; (2) invocar el servicio `monitor/get_proposal` con el dominio, el problema vigente y una observación en lenguaje natural; (3) recibir la propuesta estructurada, formada por clasificación y *deltas* de predicados; (4) si la propuesta es estructural y semánticamente válida y la política del escenario lo permite, aplicarla a través del *problem expert* de PlanSys2; y (5) solicitar a PlanSys2 una nueva planificación sobre el problema actualizado. Esta separación entre el módulo, genérico y reutilizable, y el integrador, específico del caso de uso, es una decisión de diseño deliberada: mantiene la propuesta del módulo libre de la política de aceptación, que es responsabilidad de quien lo integra.

4.5. Gestión de eventos, contexto y estado

La gestión del contexto es uno de los elementos centrales del diseño. El sistema debe decidir qué información resulta relevante para el modelo de lenguaje y cómo presentarla de forma clara, compacta y no ambigua. Un contexto excesivamente grande necesita muchos recursos y puede introducir ruido, mientras que un contexto demasiado reducido puede impedir que el modelo genere una respuesta útil. Por ello, se tienen que utilizar técnicas u algoritmos que permitan una gestión acorde para cada caso.

Desde el punto de vista de diseño, el contexto que recibe el monitor se limita a cuatro fuentes: dominio PDDL, problema vigente, observación proporcionada por la aplicación integradora y resumen de eventos de ejecución. El módulo no pretende inferir por sí solo el estado completo del mundo, sino transformar esa información en una propuesta estructurada de cambios. Los eventos de ejecución se resumen para conservar las transiciones relevantes y reducir ruido, mientras que la observación actúa como punto de especialización del caso de uso.

La responsabilidad del monitor termina en la generación y validación estructural de una propuesta: clasificación, predicados a eliminar, predicados a añadir e instancias nuevas si procede. La validación semántica completa, la aceptación de la propuesta y su aplicación sobre el `problem_expert` recaen en el nodo integrador. Esta frontera de responsabilidades evita que una respuesta del modelo modifique directamente el estado de planificación.

4.6. Interfaz con modelos LLM

La interfaz con los modelos de lenguaje debe abstraer los detalles concretos de cada proveedor o *backend* utilizado. Para ello, el diseño separa el núcleo del monitor de las implementaciones concretas de inferencia mediante una arquitectura basada en plugins. El nodo servidor carga los plugins configurados y los invoca a través de una interfaz común, sin asumir internamente qué modelo o mecanismo de ejecución se utiliza.

El paquete `plansys2_llama_monitor` constituye una implementación concreta de esta interfaz para un *backend* basado en `llama_ros`, descrito en el Capítulo 3 como *wrapper* de `llama.cpp` para ROS 2. Sin embargo, la arquitectura permite incorporar otros plugins para diferentes modelos o proveedores sin modificar la lógica principal del monitor. Esta separación facilita la comparación entre modelos, la evolución del sistema y la sustitución del *backend* LLM en función de los requisitos de cada escenario.

El plugin de referencia integra un modelo pequeño cuantizado ejecutado localmente mediante `llama_ros`. La especialización para cada despliegue no se realiza sobre los pesos del modelo, sino sobre el contexto: la observación enviada a través del cliente permite a la aplicación integradora introducir información específica de su dominio, manteniendo el modelo y el plugin

sin cambios.

4.7. Decisiones de diseño y alternativas descartadas

Durante el diseño se consideraron diferentes alternativas. Una opción era modificar directamente componentes internos de PlanSys2, pero esta alternativa reducía la modularidad y dificultaba la integración. Otra opción era delegar completamente la generación del nuevo plan en el LLM, pero esta posibilidad se descartó por falta de garantías formales y por los riesgos asociados a respuestas no verificables.

La alternativa adoptada busca un equilibrio entre flexibilidad y control: el LLM actúa como apoyo contextual, mientras que el sistema mantiene mecanismos explícitos de validación y planificación formal.

Una tercera familia de decisiones surgió de la integración real con un sistema de planificación en ejecución. Una primera versión cargaba el modelo de lenguaje mediante un proceso efímero por consulta; el coste de carga del modelo en cada llamada lo hacía inviable para un bucle de replanificación, por lo que se adoptó un *backend* persistente que se inicia una sola vez mediante el parámetro `pre_launch`. En segundo lugar, dado que la inferencia de un modelo local tiene una duración poco predecible, el orquestador acota cada consulta con un *timeout* explícito y un mecanismo de cancelación que termina el subprocesso de inferencia, en lugar de bloquear el sistema o devolver resultados parciales. Por último, se adoptó una política de fallo explícito y verificable: ante cualquier desviación del comportamiento esperado, el módulo informa de un error controlado antes que arriesgar la integridad del estado de PlanSys2. Estas decisiones, detalladas en el Capítulo 5, priorizan la robustez del sistema integrado sobre la latencia del caso favorable.

Capítulo 5

Implementación

Este capítulo describe cómo se ha materializado en código el diseño expuesto en el Capítulo 4. Se presentan primero la organización del repositorio y los componentes principales implementados (Secciones 5.1 y 5.2), seguidos de la integración en un workspace de ROS 2 (Sección 5.3). El núcleo del capítulo se centra en tres aspectos donde la implementación introduce decisiones técnicas relevantes: la generación del *prompt* y el control del contexto (Sección 5.4), incluida la contribución a *llama_ros* para reutilizar la KV-cache; el procesamiento de las respuestas del modelo (Sección 5.5); y la gestión de errores y casos límite (Sección 5.7), que materializa el principio de fallar de forma explícita y verificable.

El capítulo cierra con la validación de propuestas (Sección 5.6), donde se delimita qué parte del control queda dentro del módulo y qué parte se delega en el nodo integrador.

5.1. Estructura del repositorio/submódulo

El módulo desarrollado se organiza como un submódulo independiente que puede descargarse, compilarse e integrarse en un workspace de ROS 2. Esta organización facilita su reutilización y permite mantener separado el desarrollo del módulo respecto al código base de PlanSys2.

```
1 plansys2_llm_monitor/  
2   |-- LICENSE  
3   |-- README.md  
4   |-- plansys2_monitor_msgs/      # ROS 2 interfaces  
5   |-- plansys2_monitor/           # Nucleo del monitor  
6   |-- plansys2_llama_monitor/     # Plugin LLM (llama_ros)  
7 plansys2_llm_examples/  
8   |-- ...                         # Ejemplo de integracion
```

Listing 5.1: Estructura orientativa del repositorio

El código del módulo y del demostrador se distribuye como software libre bajo licencia Apache-2.0. Las URL de los repositorios y la información de las ramas se recogen en el Apéndice A.

5.2. Componentes principales desarrollados

Los componentes principales implementados se corresponden uno a uno con los bloques B1–B6 del diseño (Sección 4.2). La Tabla 5.1 resume su responsabilidad, el módulo o nodo ROS 2 en el que residen y la salida que producen. Como se anticipó en la Sección 4.2, B1–B5 viven dentro del nodo `MonitorNode` del paquete `plansys2_monitor`, mientras que B6 corresponde al cliente C++ que se enlaza en la aplicación integradora. La interfaz pública del módulo se reduce, por tanto, a un único servicio ROS 2 (`monitor/get_proposal`) y un único mensaje de respuesta (`plansys2_monitor_msgs/Proposal`).

Componente	Uso	Salida
B1	Captura los eventos de ejecución de PlanSys2 y mantiene la traza histórica.	Fichero <code>monitor_action_hub.txt</code> en <code>/tmp/<ns>/</code> .
B2	Carga los plugins LLM, expone el servicio ROS 2 y orquesta la ejecución en paralelo.	Nodo <code>monitor</code> , servicio <code>monitor/get_proposal</code> y mensajes <code>ProposalArray</code> .
B3	Ensambla el <i>prompt</i> a partir del dominio, el problema, el resumen del log y la observación.	Cadena de texto con el <i>prompt</i> y el esquema JSON de salida obligatorio.
B4	Ejecuta la inferencia mediante el <i>backend</i> LLM cargado como plugin.	Subproceso <code>ros2 llama prompt</code> y respuesta cruda del LLM.
B5	Convierte la respuesta cruda en un mensaje ROS 2 tipado.	Mensaje <code>plansys2_monitor_msgs/msg/Proposal</code> con clasificación y predicados.
B6	Cliente C++ que invoca el servicio desde los nodos de aplicación.	<code>std::optional<Proposal></code> con la propuesta o <code>nullopt</code> si falla.

Tabla 5.1: Componentes del módulo

5.3. Integración y compilación

El módulo se integra dentro de un workspace de ROS 2 mediante el sistema de compilación habitual basado en `colcon`. La compilación debe resolver las dependencias declaradas en el paquete y generar los ejecutables o librerías necesarios para su uso en los escenarios de prueba.

```
1 cd ~/ros2_ws
2 colcon build --symlink-install # GPU: --cmake-args -DGGML_CUDA=ON
3 source install/setup.bash
```

Listing 5.2: Compilación del módulo en un workspace ROS 2

5.4. Generación de prompts y control del contexto

La estrategia adoptada para que el modelo de lenguaje pueda razonar sobre un fallo de ejecución consiste en construir, en cada invocación, un *prompt* que combina una plantilla fija con una cabecera constante, formada por el rol, las reglas y el esquema JSON de salida, seguida de cuatro fuentes de contexto en orden de volatilidad creciente: el dominio PDDL, la observación, el estado del problema y el resumen del registro de acciones. Esta composición se realiza en la función estática `makePrompt()`, definida en la clase base `MonitorBase` y reutilizada por cualquier *plugin* de *backend* LLM. Con ello se cubren RF-03 y OE6: la construcción del *prompt* es reproducible, estructurada y común a todos los *backends*.

5.4.1. Control del contexto: resumen del registro de acciones

El registro de eventos generado por el componente **B1**, descrito en la Tabla 5.1, se almacena en el fichero `monitor_action_hub.txt` y puede contener decenas de entradas por cada acción ejecutada. Insertarlo íntegramente en el *prompt* desplazaría al dominio y al problema PDDL fuera de la ventana de contexto admisible por el modelo. Por ello, antes de su inyección, el registro se procesa mediante la función `summarizeActionLog()`, que aplica una compresión semántica configurable por el parámetro `summarize_mode`.

- **limited**: modo por defecto. Conserva únicamente las entradas terminales del ciclo de ejecución, es decir, los eventos de tipo `FINISH` y `CANCEL`. Estos contienen el resultado de cada acción intentada y descartan el ruido de transiciones intermedias como `REQUEST`, `RESPONSE` o `FEEDBACK`.
- **full**: conserva todas las transiciones, anteponiendo el tipo del evento entre corchetes. Este modo se reserva para depuración y para escenarios en los que el ciclo completo de negociación de una acción es relevante para el razonamiento del modelo.

En ambos modos, cada entrada se colapsa a una línea con un formato compacto que indica la acción, sus argumentos, el resultado, SUCCESS o FAILED, y, opcionalmente, el mensaje de estado asociado.

```
1 move(robot1 room_a room_b): SUCCESS.
2 pick(robot1 cup): FAILED. object not reachable
```

Listing 5.3: Formato del resumen del registro de acciones en modo limited

5.4.2. Plantilla del prompt

La plantilla utilizada por `makePrompt()` se compone de dos partes con propiedades distintas respecto a su variación entre invocaciones. Una **cabecera constante** fija el papel del modelo, enuncia las reglas semánticas y declara el esquema JSON de salida obligatorio; este bloque es idéntico en todas las llamadas. A continuación se inserta el **cuerpo variable**, formado por las cuatro fuentes de contexto, ordenadas de menor a mayor volatilidad: el dominio PDDL, estable durante toda la ejecución; la observación redactada por el integrador; el estado del problema; y, en último lugar, el resumen del registro de acciones, que es la fuente que más cambia y que crece a lo largo de la tarea.

Tres decisiones metodológicas guían el diseño:

- **Reglas semánticas explícitas dentro del propio prompt**, sin depender de *fine-tuning* ni de comportamientos emergentes del modelo. El *prompt* declara, por ejemplo, que no deben modificarse predicados de objetos no mencionados en la observación.
- **Esquema JSON de salida fijado en la cabecera**, de modo que la respuesta del modelo sea directamente *parseable* por código determinista. Este esquema convierte la tarea de procesamiento de la respuesta, descrita en la Sección 5.5, en un *parsing* estructural y no en una interpretación de lenguaje natural.
- **Ordenación estático-primero / dinámico-último**, que no es estética sino un contrato de rendimiento sobre la caché de claves-valores, o KV-cache, del modelo, detallado en la Sección 5.4.4.

```
1 You are a PDDL state update assistant. You will receive a PDDL domain
  ,
2 a task description with observations, the problem state, and an
3 action execution log. Determine what state changes are needed.
4
5 RULES:
6 - Only change predicates directly affected by the observation
```



```

7 - Do NOT change predicates for objects not mentioned
8 - A fact's name and arguments must match a Domain (:predicates)
  signature exactly - never the failed action's arguments or actor
9 - If an object moved from A to B: remove (object_at obj A), add (
  object_at obj B)
10 - If no changes are needed, classify as CORRECT
11
12 Reply ONLY with a JSON object in this exact format:
13 {
14   "classification": "MODIFY_PLAN" or "CORRECT" or "UNSOLVABLE",
15   "reasoning": "brief explanation",
16   "remove_predicates": ["(predicate1)", "(predicate2)"],
17   "add_predicates": ["(predicate1)", "(predicate2)"],
18   "add_instances": [],
19   "domain_changes": []
20 }
21
22 --- Domain ---
23 <contenido del dominio PDDL>
24
25 --- Task & Observations ---
26 <observacion en lenguaje natural>
27
28 --- Problem ---
29 <contenido del problema PDDL>
30
31 --- Action execution log ---
32 <resumen producido por summarizeActionLog()>

```

Listing 5.4: Plantilla del prompt construido por makePrompt()

El parámetro booleano `prompt_debug`, cuando está activo, vuelca el *prompt* completo a un fichero de inspección. Esto facilita la depuración del razonamiento del modelo sin necesidad de instrumentación adicional.

5.4.3. Esquema JSON de salida

El esquema JSON al que debe ajustarse la respuesta del modelo se declara en la **cabecera del prompt**, en el bloque inmediatamente posterior a las reglas y antes del dominio PDDL. Esta posición es deliberada: sitúa la especificación de salida dentro de la parte invariante del *prompt*, lo que permite su reutilización en la KV-cache, como se describe en la Sección 5.4.4. El esquema

fija:

- Un campo `classification` de tipo cadena. El *prompt* restringe la respuesta del modelo a tres valores: `CORRECT`, `MODIFY_PLAN` y `UNSOLVABLE`. El mensaje `plansys2_monitor_msgs/Proposal` define además las constantes `MODIFY_DOMAIN` y `ERROR`; esta última no es un valor que el modelo deba producir, sino el centinela que `parseResponse()` asigna ante cualquier respuesta no interpretable, como se detalla en la Sección 5.5.
- Un campo `reasoning` de texto libre, destinado exclusivamente a depuración y trazabilidad; no es consumido por el procesamiento posterior.
- Cuatro listas tipadas que describen las modificaciones propuestas al estado: `remove_predicates`, `add_predicates`, `add_instances` y `domain_changes`.

La elección de un esquema cerrado, en lugar de permitir al modelo proponer la transformación en formato libre, responde a dos objetivos: garantizar que la salida sea verificable de forma determinista por código y acotar el espacio de respuesta para reducir alucinaciones. El procedimiento concreto que extrae estos campos de la respuesta cruda del modelo y los traduce a un mensaje ROS 2 tipado se describe en la Sección 5.5.

5.4.4. Orden del prompt y reutilización de la KV-cache

El orden de los bloques dentro del *prompt* obedece a un contrato de rendimiento, no a una preferencia de presentación. Los modelos de lenguaje autorregresivos calculan, durante la fase de evaluación del *prompt*, una caché de claves y valores, o KV-cache, cuyo coste es proporcional a la longitud del prefijo. Cuando dos invocaciones consecutivas comparten un prefijo idéntico, ese prefijo no necesita recomputarse: la caché se reutiliza y solo se evalúa la parte final que ha cambiado.

El módulo explota esta propiedad situando todo el contenido invariante al principio del *prompt*: el rol, las reglas y el esquema JSON de salida, seguido de las fuentes de contexto en orden de volatilidad creciente. El dominio PDDL permanece estable entre replanificaciones, mientras que el resumen del registro de acciones cambia y crece en cada ciclo; por ello el dominio se coloca lo antes posible y el registro de acciones estrictamente al final. Mover el esquema JSON al final del *prompt*, una disposición intuitiva ya que describe la salida esperada, invalidaría esta reutilización al introducir un bloque constante después de los datos variables.

El efecto de esta ordenación es una reducción sustancial del número de tokens que deben recomputarse en cada invocación, lo que se traduce directamente en menor latencia de inferencia. Este impacto es especialmente relevante al ejecutar modelos pequeños sobre hardware embarcado con recursos limitados; su cuantificación experimental se desarrolla en el Capítulo 6.

5.4.5. Contribución a llama_ros: reutilización de KV-cache

La ordenación del *prompt* descrita en la Sección 5.4.4 solo produce una ganancia medible si el *backend* de inferencia reutiliza efectivamente la caché de claves-valores entre llamadas consecutivas. Esta reutilización no la implementa directamente el módulo de este trabajo, sino el *backend* llama_ros. Durante el desarrollo se detectó que el servidor de inferencia descartaba la caché de claves-valores en cada petición, recomputando el *prompt* completo, lo que hacía inviable un bucle de replanificación sobre hardware embarcado. Para resolverlo, se desarrolló una modificación de llama_ros que preserva el estado de la caché entre peticiones consecutivas al mismo *slot*: identifica el prefijo de *tokens* común más largo entre el *prompt* cacheado y el nuevo, y recomputa únicamente el sufijo divergente. Esta modificación, junto con el arranque persistente del servidor (*pre_launch*), fue contribuida al proyecto original mediante el *pull request* «feat(llama): reuse KV-cached prefix between consecutive requests» (PR n.º 38, https://github.com/mgonzs13/llama_ros/pull/38), revisada y aceptada en la rama principal de llama_ros (commit de fusión 7b93280).

El reparto de responsabilidades es, por tanto, el siguiente: llama_ros, con la contribución descrita, provee el mecanismo de reutilización de prefijo; el módulo de este trabajo provee el contrato que lo hace explotable, situando el contenido invariante al principio del *prompt* y el variable al final. La ganancia conjunta se cuantifica empíricamente sobre hardware embarcado en el Capítulo 6.

Para ilustrar el contenido real del *prompt* sin sobrecargar la memoria, esta sección reproduce únicamente el bloque más informativo: la observación textual redactada por el integrador a partir de la percepción YOLO. El *prompt* completo del demostrador, con el dominio PDDL, el problema vigente y la traza de acciones, alcanza unos 3 050 *tokens* (155 líneas) y se conserva íntegro en el repositorio del proyecto; el parámetro `prompt_debug=true` lo vuelca a fichero, como se indica en el Apéndice A. El Listing 5.5 reproduce el bloque de tarea y observaciones de una ejecución del caso simple, y el Listing 5.6 muestra la respuesta cruda del modelo Qwen2.5-3B sobre ese mismo *prompt*.

```
1 --- Task & Observations ---
2 A robot action just failed: the world is not as the PDDL problem
   believes.
3 The perception list below is GROUND TRUTH about the objects the robot
   saw.
4 Your only job: make the problem's facts about those objects match
   perception.
5 This is belief correction, NOT planning - never reason about where an
   object
6 should go or what it should be, only what perception says it IS now.
7
```

```

8 The failed action was pick_book(curiosity, red_book, shelf_red).
9 The objects it names are the ones whose beliefs are most likely wrong
10 .
11 Perception:
12 - red book -> middle_path

```

Listing 5.5: Bloque de tarea y observaciones, extracto del prompt real

```

1 {
2   "classification": "MODIFY_PLAN",
3   "reasoning": "The pick_book action failed because the robot could
4     not find the red_book at shelf_red. The perception shows the
5     red_book is at the middle_path.",
6   "remove_predicates": ["(object_at red_book shelf_red)"],
7   "add_predicates":    ["(object_at red_book middle_path)"],
8   "add_instances":     [],
9   "domain_changes":    []
10 }

```

Listing 5.6: Respuesta del modelo Qwen2.5-3B

El nodo integrador recibe esta respuesta, la valida estructuralmente (clasificación dentro del conjunto admisible, predicados como cadenas), comprueba que `object_at` está declarado en el dominio y que `red_book` y `middle_path` son objetos conocidos, y aplica los *remove/add* sobre el `problem_expert` antes de solicitar una nueva planificación a POPF.

5.5. Procesamiento de respuestas del LLM

Una vez que el modelo de lenguaje produce su respuesta, el módulo debe traducirla a una estructura tipada que el resto del sistema pueda consumir. Dado que el *prompt* fuerza al modelo a contestar conforme al esquema JSON descrito en la Sección 5.4.3, esta tarea se reduce a un *parsing* estructural determinista, sin necesidad de procesamiento de lenguaje natural. Toda la lógica se concentra en la función estática `parseResponse()` de la clase base `MonitorBase`, que devuelve directamente un mensaje `plansys2_monitor_msgs/Proposal` listo para ser utilizado por el resto del sistema.

5.5.1. Extracción tolerante del bloque JSON

Aunque el *prompt* indica al modelo que responda *únicamente* con un objeto JSON, en la práctica algunos modelos pueden prefijar o sufijar la respuesta con texto explicativo. Por ejemplo, una

respuesta puede comenzar con una frase auxiliar antes del objeto JSON. Forzar una coincidencia exacta provocaría errores innecesarios. Por ello, el procedimiento de extracción es deliberadamente tolerante: localiza la primera aparición del carácter { y la última aparición del carácter } en la respuesta cruda, y considera como candidato a JSON la subcadena comprendida entre ambos índices.

```
1 const auto json_start = raw_response.find('{');
2 const auto json_end = raw_response.rfind('}');
3
4 if (json_start == std::string::npos ||
5     json_end == std::string::npos ||
6     json_end <= json_start) {
7     return solution; // queda como ERROR
8 }
9
10 const std::string json_str =
11     raw_response.substr(json_start, json_end - json_start + 1);
```

Listing 5.7: Extracción tolerante del objeto JSON en parseResponse()

Esta estrategia absorbe el ruido habitual del modelo sin renunciar al control sobre la estructura de la respuesta. Si el bloque candidato no es JSON válido, el *parser* de la biblioteca `nlohmann::json` lanzará una excepción, que se gestiona dentro del propio procedimiento de conversión.

5.5.2. Validación estructural y traducción al mensaje ROS 2

El bloque extraído se interpreta con `nlohmann::json::parse` y se valida campo a campo antes de poblar el mensaje `plansys2_monitor_msgs/Proposal`. La validación se realiza en dos etapas principales:

1. **Campo obligatorio `classification`:** debe existir, ser una cadena y pertenecer al conjunto cerrado formado por `CORRECT`, `MODIFY_PLAN`, `MODIFY_DOMAIN`, `UNSOLVABLE` o `ERROR`. Cada valor se traduce a la constante correspondiente del mensaje ROS 2. Cualquier desviación, como un campo ausente, un tipo incorrecto o un valor fuera del conjunto, provoca que el mensaje conserve la clasificación inicial `ERROR`.
2. **Listas opcionales:** los campos `add_predicates`, `remove_predicates`, `add_instances` y `domain_changes` se procesan únicamente si están presentes y son arrays. Cada elemento que no sea una cadena se descarta. La ausencia de una lista no se considera un error, ya que muchas respuestas válidas, como una clasificación `CORRECT`, no requieren modificaciones de estado.

Adicionalmente, el campo **resolution** del mensaje devuelto conserva la respuesta cruda del modelo. Esto permite que los componentes aguas abajo, incluido el cliente de integración, inspeccionen la respuesta original del LLM en caso de necesitarla para depuración o auditoría.

5.6. Validación de propuestas antes de la replanificación

La validación es una parte fundamental del sistema porque impide que una salida generada por el modelo de lenguaje modifique directamente el estado gestionado por PlanSys2. Una respuesta del LLM no se considera correcta por estar bien redactada ni por contener un JSON sintácticamente válido: debe superar comprobaciones explícitas antes de que el cliente integrador pueda usarla para actualizar el problema de planificación.

En la implementación actual se distinguen tres niveles de control, pero no todos pertenecen al mismo componente. El primer nivel es **estructural**: la respuesta debe contener un objeto JSON reconocible, el campo obligatorio **classification** debe pertenecer al conjunto admitido y las listas de cambios deben estar formadas por cadenas. Este nivel sí se implementa dentro del propio monitor, tal como se describe en la Sección 5.5.2, y cubre la parte automatizada de RF-05 y OE7 dentro del módulo.

El segundo nivel es **semántico**: los predicados, instancias y posibles cambios de dominio propuestos deben contrastarse con el dominio PDDL y con el problema vigente. Esta comprobación se delega en el componente que integra el monitor con PlanSys2, ya que dispone del estado actualizado y puede decidir si una propuesta es compatible con los objetos, tipos, predicados y restricciones del dominio.

El tercer nivel es la **aceptación operativa**: incluso cuando una propuesta es estructural y semánticamente válida, el sistema debe decidir si conviene aplicarla y solicitar una nueva planificación. Esta decisión depende de la observación recibida, del estado de ejecución, del objetivo activo y de las políticas de seguridad del escenario. Por tanto, el LLM queda limitado a producir hipótesis de actualización, mientras que la modificación del problema y la llamada final al planificador permanecen bajo control explícito del integrador. En la trazabilidad del Capítulo 8, RF-05 y OE7 se consideran cumplidos de forma parcial precisamente por este reparto de responsabilidades.

5.7. Gestión de errores y casos límite

El módulo opera contra un *backend* cuya salida no puede garantizarse a priori: un modelo de lenguaje puede responder con formato incorrecto, tardar más de lo aceptable o no responder en absoluto. Por ello, el diseño se rige por el principio de **fallar de forma explícita y verificable**:

ante cualquier desviación del comportamiento esperado, el sistema informa de un fallo controlado antes que producir un resultado degradado que pudiera comprometer la integridad del estado de PlanSys2.

Esta filosofía se materializa en una política unificada de propagación de errores. Cualquier fallo, sea estructural, operacional o de infraestructura, conduce al mismo resultado externo: un mensaje `Proposal` con clasificación `ERROR` y un código de *status* del servicio `GetProposal` igual a `ERROR`, acompañado de un campo `error_info` que describe la causa. No se realizan reintentos silenciosos ni se devuelven resultados parciales: la decisión de reintentar, abortar o adoptar una estrategia alternativa se delega siempre en el cliente que invocó al monitor.

5.7.1. Errores en la respuesta del modelo

Se agrupan en esta categoría los fallos detectados por `parseResponse()`, descrita en la Sección 5.5, al procesar la salida cruda del LLM. Los casos cubiertos son:

- Respuesta vacía, generalmente por cancelación del subproceso al agotarse el *timeout* de inferencia.
- Respuesta sin un objeto JSON identificable, es decir, sin pares de llaves `{...}` reconocibles.
- Objeto JSON sintácticamente válido pero sin el campo obligatorio `classification`.
- Campo `classification` presente con un valor fuera del conjunto admisible: `CORRECT`, `MODIFY_PLAN`, `MODIFY_DOMAIN`, `UNSOLVABLE` o `ERROR`.
- Listas de predicados o instancias mal formadas, ya sea por no ser arrays o por contener elementos que no son cadenas.

Cualquier escenario no enumerado en esta lista —por ejemplo una clasificación con tipo correcto pero valor inesperado, o un campo presente con tipo de datos incoherente— se traduce, por construcción del flujo de control de `parseResponse()`, en una clasificación final `ERROR` con `error_info` descriptivo. El módulo nunca devuelve una propuesta parcial o ambigua: o cumple el esquema completo, o falla de forma explícita.

En todos los casos, el mensaje `Proposal` conserva el campo `resolution` con la respuesta cruda del modelo. Esto permite realizar una auditoría posterior de la salida del LLM sin necesidad de instrumentación adicional.

5.7.2. Errores operacionales: *timeouts* y cancelación

La inferencia contra un modelo de lenguaje local puede prolongarse de forma impredecible. Para acotar la latencia del servicio, el orquestador, correspondiente al componente **B2** de la Tabla 5.1,

impone un *timeout* global configurable mediante el parámetro `propose_timeout`, con un valor por defecto de 240 s.

El control de tiempo se implementa mediante `std::async` sobre cada *plugin* activo y un bucle de espera con sondeo no bloqueante:

- Si un *plugin* entrega su resultado antes del *timeout*, su salida se acumula para construir la respuesta agregada del servicio.
- Si el *timeout* expira con *plugins* todavía en ejecución, el orquestador invoca `cancel()` sobre cada uno de ellos y espera 100 ms adicionales para que la cancelación se propague. El *plugin* `LLAMAMonitor` observa la solicitud de cancelación en su bucle de sondeo del *pipe* de `stdout` y envía `SIGTERM` al subproceso del LLM, evitando que queden recursos colgados tras la cancelación.
- Si todos los *plugins* son cancelados sin entregar resultado, el servicio responde con `status = ERROR` y `error_info = Resolution not found`.

De forma análoga, el componente `MonitorClient`, correspondiente al bloque **B6**, aplica un *timeout* propio al esperar la respuesta del servicio, con un valor por defecto de 300 s. Si dicho tiempo expira o si el servicio no está disponible, el cliente devuelve `std::nullopt`.

5.7.3. Errores de configuración y de infraestructura

Los errores detectables en tiempo de arranque se tratan conforme al patrón *fail-fast*, abortando la transición del ciclo de vida del nodo antes de aceptar peticiones. Los casos principales son:

- La imposibilidad de crear el directorio o el fichero de registro en `/tmp/<ns>/` provoca el rechazo de la transición `on_configure`, devolviendo `CallbackReturn::FAILURE`.
- Una excepción de `pluginlib` durante la carga de un *plugin* declarado en `monitor_plugins` se considera un fallo no recuperable y termina el proceso.
- Una lista de *plugins* vacía produce un fallo de configuración explícito, evitando que el nodo arranque en un estado inválido en el que el servicio exista pero no haya ningún *backend* capaz de atenderlo.

Este tratamiento temprano de los errores evita que el módulo quede activo en un estado parcialmente inicializado o incoherente con su configuración.

5.7.4. Casos límite contemplados

Además de los errores propiamente dichos, el módulo contempla varios escenarios límite que no constituyen necesariamente un fallo, pero que requieren un tratamiento explícito:

- **Registro de acciones vacío:** cuando el monitor se invoca antes de que el *executor* de PlanSys2 haya emitido eventos, por ejemplo en el primer ciclo de planificación, la función `summarizeActionLog()` devuelve el literal (`no action log available`). Esta señal explícita evita que el modelo interprete un campo vacío como ausencia implícita de eventos relevantes.
- **Inundación de mensajes FEEDBACK:** el callback de la suscripción al *topic* `actions_hub` aplica un filtro de progreso sobre mensajes consecutivos del mismo nodo y tipo. En concreto, se descartan actualizaciones redundantes cuando $\Delta completion < 0,30$, reduciendo el ruido generado por acciones largas.
- **Pre-lanzamiento del *backend* LLM:** cuando el parámetro `pre_launch` está activo, el subproceso del modelo se inicia durante `on_configure` para evitar la latencia de arranque en cada petición. Si este subproceso muere entre invocaciones, el *plugin* lo relanza de forma transparente en la siguiente llamada a `getProposal()`, sin propagar el evento al cliente como un error externo.

En conjunto, esta política convierte el módulo en un componente robusto frente a la indeterminación característica de los modelos de lenguaje, sin sacrificar la integridad del estado de planificación gestionado por PlanSys2.

5.8. Ejemplo de uso del módulo

El módulo se integra en un demostrador robótico completo. Un robot Kobuki simulado opera en el mundo *bookstore* de AWS RoboMaker bajo Gazebo, con navegación autónoma proporcionada por EasyNav, percepción mediante YOLO a través de `yolo_ros` cuando se usa `perception_mode:=real`, y ejecución de acciones simbólicas mediante árboles de comportamiento de PlanSys2 (`move`, `pick_book`, `place_book`). El demostrador también permite modos de percepción sintética para pruebas controladas. Sobre esta pila, PlanSys2 con POPF genera el plan y el módulo de este trabajo actúa como capa de replanificación cuando la percepción contradice el estado simbólico. El nodo `reception_controller_node` funciona como integrador: detecta la incidencia, invoca el servicio del monitor, valida la propuesta recibida y actualiza el problema antes de solicitar una nueva planificación. El ejemplo siguiente describe el ciclo de replanificación reactiva de extremo a extremo sobre este sistema; su evaluación cuantitativa se desarrolla en el Capítulo 6.

Para ilustrar el funcionamiento integrado de los componentes descritos en este capítulo, se presenta a continuación un escenario representativo de replanificación reactiva. La descripción se mantiene a nivel funcional; el análisis cuantitativo y la evaluación experimental sobre este mismo escenario se desarrollan en el Capítulo 6.

Escenario

Un robot Kobuki opera en una librería con la tarea de recoger un libro localizado en una estantería y depositarlo sobre una mesa determinada. El plan inicial generado por PlanSys2 consta de cuatro acciones principales: desplazarse hasta la estantería de origen, recoger el libro, desplazarse a la mesa de destino y depositarlo sobre ella.

Durante la ejecución, el sistema de percepción detecta que el libro no se encuentra en la estantería esperada, sino en otra distinta. Esta observación invalida el plan en curso, ya que el estado simbólico del problema sigue representando el libro en su ubicación inicial. El nodo integrador solicita entonces una replanificación al módulo `plansys2_llm_monitor`.

Ciclo de replanificación

El ciclo completo, desde la detección de la incidencia hasta la reanudación del plan, se desarrolla en cinco etapas:

1. **Captura del contexto.** El componente **B1** registra en segundo plano los eventos de ejecución emitidos por el *executor* de PlanSys2. En el momento de la incidencia, el registro contiene la transición terminal del primer desplazamiento, con resultado SUCCESS, y la del intento de recogida, con resultado FAILED.
2. **Invocación del servicio.** El nodo integrador utiliza el cliente del bloque **B6** para invocar el servicio `monitor/get_proposal`. La petición incluye el dominio PDDL vigente, el problema actual y una observación en lenguaje natural que describe la nueva ubicación detectada para el libro.
3. **Generación del *prompt*.** El bloque **B3** resume el registro de acciones en modo `limited`, conservando únicamente los eventos terminales, y ensambla el *prompt* con las cuatro fuentes de contexto descritas en la Sección 5.4.
4. **Inferencia y procesamiento de la respuesta.** El bloque **B4** ejecuta la inferencia sobre el modelo local y devuelve una respuesta cruda. A continuación, el bloque **B5** extrae el objeto JSON, valida su estructura y lo traduce a un mensaje `Proposal`. En este caso, la respuesta se clasifica como `MODIFY_PLAN` y propone sustituir el predicado que ubicaba al libro en la estantería original por otro que refleja su nueva ubicación.

5. **Aplicación y replanificación.** El nodo integrador aplica las modificaciones aceptadas sobre el `problem_expert` de PlanSys2. Una vez actualizado el problema, se solicita un nuevo plan. El plan resultante incluye un desplazamiento a la estantería en la que realmente se encuentra el libro, tras lo cual se completa la entrega sobre la mesa de destino.

La Figura 5.1 resume gráficamente el flujo. El resultado funcional del ciclo, así como la evaluación de su latencia, precisión del modelo y robustez frente a perturbaciones, se detalla en el Capítulo 6.

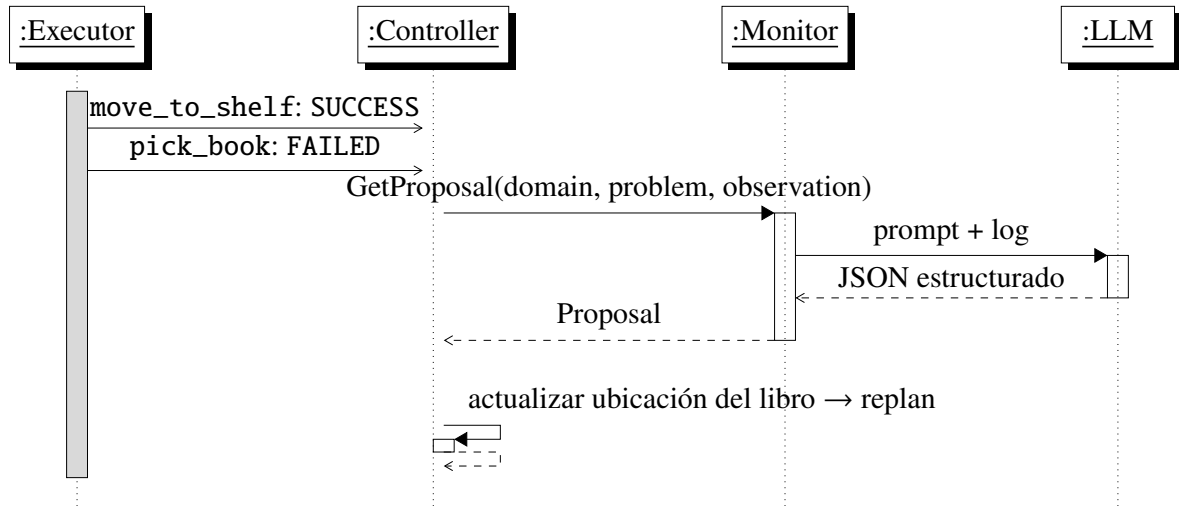


Figura 5.1: Ciclo de replanificación

Capítulo 6

Experimentación y validación

Este capítulo da respuesta al objetivo OE8 mediante un escenario robótico representativo y una comparativa entre modelos. Evalúa empíricamente el módulo propuesto. La validación tiene dos planos complementarios: una **comparativa sistemática de modelos de lenguaje pequeños** sobre la tarea de corrección de estado PDDL, y la **viabilidad de despliegue** del modelo seleccionado sobre hardware representativo de un robot de servicio de bajo coste. El segundo plano cierra el hilo argumental abierto en el Capítulo 5: la decisión de diseño sobre el orden del *prompt* (Sección 5.4.4) se valida aquí con medidas reales.

6.1. Escenario de prueba y caso de uso

La validación se articula sobre el demostrador `bookstore_kobuki`: un robot Kobuki que opera en una librería con la tarea de recoger un libro de una estantería y depositarlo en una mesa. Durante la ejecución, el sistema de percepción detecta que el libro objetivo no se encuentra en la estantería esperada, lo que invalida el plan en curso y dispara una solicitud de replanificación al módulo. El escenario se lanza con:

```
ros2 launch plan_bookstore bookstore_kobuki_launch.py \
  perception_mode:=real displaced_book:=red_book
ros2 run plan_bookstore reception_controller_node --ros-args \
  --params-file src/planning/plansys2_llm_examples/params/planner_param.yaml \
  -p displaced_book:=red_book
```

Sobre este caso de uso se definen tres escenarios de evaluación de la corrección de estado, recogidos en la Tabla 6.1.

Caso	Situación	Cambios esperados	Resultado esperado
simple	Un objeto cambia de ubicación	1 predicado	MODIFY_PLAN, un <i>remove</i> y un <i>add</i>
complex	Varios cambios y objeto nuevo	3 predicados y 1 instancia	MODIFY_PLAN, múltiples <i>remove/add</i> y <i>add_instances</i>
edge	Observación irrelevante, sin cambio real	0	CORRECT, sin modificaciones

Tabla 6.1: Casos de replanificación

El escenario real del demostrador corresponde a un caso *simple*: el libro *red_book* detectado fuera de su estantería. El *ground truth* es *remove (object_at red_book shelf_red)*, *add (object_at red_book middle_path)* y clasificación *MODIFY_PLAN*.

6.2. Metodología experimental

La comparativa se desarrolló de forma incremental en tres versiones, cada una corrigiendo una limitación metodológica de la anterior (Tabla 6.2).

Versión	Modelos	Inferencias	Entrada
V1 (baseline)	5 (3 fallan al cargar)	~10	Texto plano REMOVE/ADD
V2	13	234 (3 escenarios × 2 modos × 3 iter.)	JSON con <i>prompt</i> sintético
V3	10	30 (3 iter.)	JSON con <i>prompt</i> real capturado del demo (YOLO)

Tabla 6.2: Versiones experimentales

El banco base es un Intel Core i7-12700H con GPU RTX 3050 Ti Laptop (4 GB VRAM), usando *llama.cpp* con *GGML_CUDA=ON* y *flash attention*. Todos los modelos se cuantizan a *Q4_K_M* salvo dos excepciones documentadas (DeepSeek-R1 en *Q8_0*, Gemma-3 en *Q4_0*). Las inferencias se realizan con *temperature = 0.0*, lo que hace las respuestas deterministas;

en consecuencia, las tres iteraciones por prueba no miden varianza estadística inferencial, sino que aíslan el efecto de calentamiento (la primera inferencia tras cargar el modelo incluye *warmup* del KV-cache y JIT de kernels CUDA, con latencia inflada). Las latencias reportadas son medianas de las iteraciones 2 y 3.

Cada inferencia se puntúa con una rúbrica de 0 a 5: 5 indica predicados y clasificación correctos contra el *ground truth*; 0 indica salida no utilizable. Las métricas empleadas para la comparativa, resumidas en la Tabla 6.3, combinan corrección funcional, latencia, consumo de memoria y modo de fallo. La evaluación incorpora dos correcciones necesarias: un **detector de no-op**, que penaliza el caso patológico en que el modelo propone `remove` y `add` del mismo predicado (una corrección semánticamente vacía), y un **extractor de JSON tolerante**, que prueba cada `{` como inicio de bloque hasta encontrar el primero parseable.

Métrica	Definición
Score (0–5)	Corrección de predicados y clasificación frente al <i>ground truth</i> .
Latencia mediana	Mediana del <i>wall clock</i> de inferencia (iteraciones 2–3).
VRAM / RSS	Memoria pico del proceso de inferencia.
Modo de fallo	Categoría del error: correcto, extracción incompleta, clasificación errónea, alucinación o JSON inválido.
Ratio CPU/GPU	Factor de ralentización al ejecutar sin GPU.

Tabla 6.3: Métricas experimentales

6.3. Modelos comparados

V2 evaluó 13 modelos por debajo de 4 000 M de parámetros (familias Qwen2.5, Phi, Llama-3.2, Gemma, SmolLM2, Falcon3, StableLM, TinyLlama, DeepSeek-R1). V3 retuvo los 10 que producían JSON válido de forma consistente en V2, descartando TinyLlama, Gemma-3 y DeepSeek-R1, que nunca generaron un objeto JSON parseable. La configuración exacta de cada modelo (repositorio GGUF, cuantización, parámetros de contexto) está documentada en el repositorio del proyecto.

6.4. Resultados con prompts sintéticos (V2)

La Tabla 6.4 recoge el ranking V2 por *score* medio sobre 18 inferencias por modelo.

Pos.	Modelo	Score	Latencia mediana
1	Llama-3.2-3B	3.33	5 472 ms
2	Gemma-2-2B	3.00	4 917 ms
3	Qwen2.5-1.5B	2.67	3 584 ms
4	Qwen2.5-3B	2.50	4 864 ms
5	Phi-4-mini	2.33	3 509 ms
6	Phi-3.5-mini	2.17	6 300 ms
11–13	TinyLlama, Gemma-3, DeepSeek-R1	0.00	— (JSON nunca válido)

Tabla 6.4: Ranking V2

La Figura 6.1 muestra la frontera de Pareto score–latencia: para aumentar el *score* hay que pagar latencia, y varios modelos de 1,5–2 B (Qwen2.5-1.5B, Gemma-2, SmolLM2) compiten en la frontera con modelos de 3 B. La Figura 6.2 desglosa el *score* por escenario y modo de log: ningún modelo supera 3 en el escenario *complex*, y el modo del log de acciones (*limited* frente a *full*) tiene un **efecto opuesto según el modelo** (Phi-4 colapsa en *full*, Llama-3.2-3B mejora). La Figura 6.3 distribuye los cinco modos de fallo y confirma que tres modelos no producen JSON jamás y que Qwen2.5 alucina más que Gemma-2 o Llama-3.2-3B.

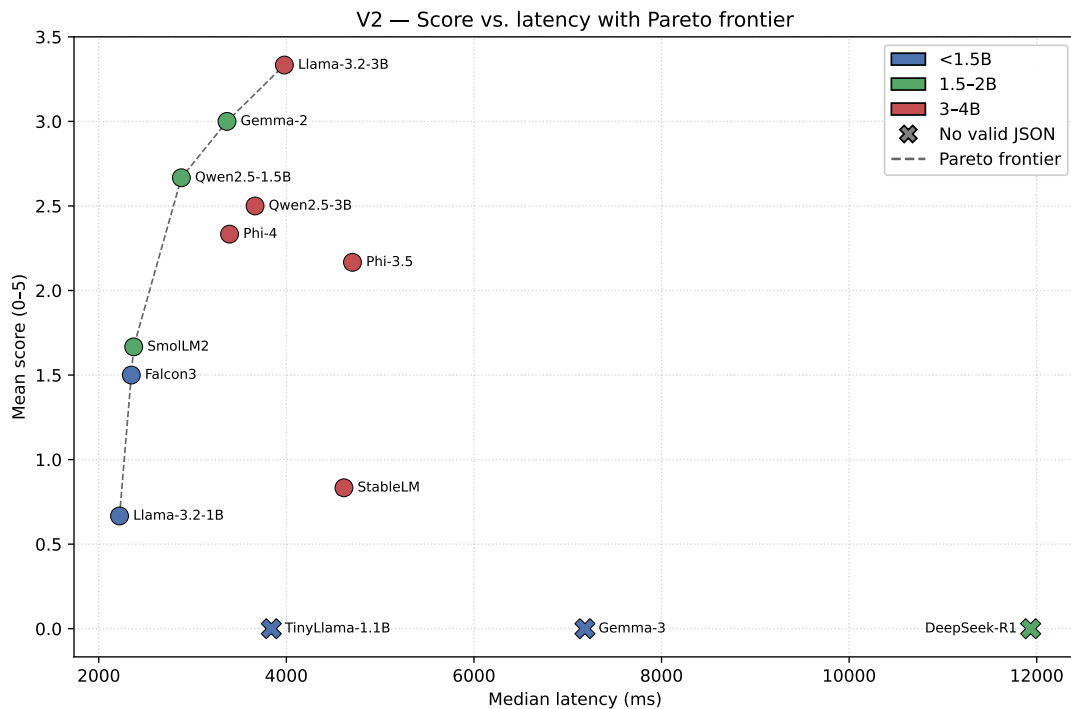


Figura 6.1: Frontera de Pareto score–latencia (V2).

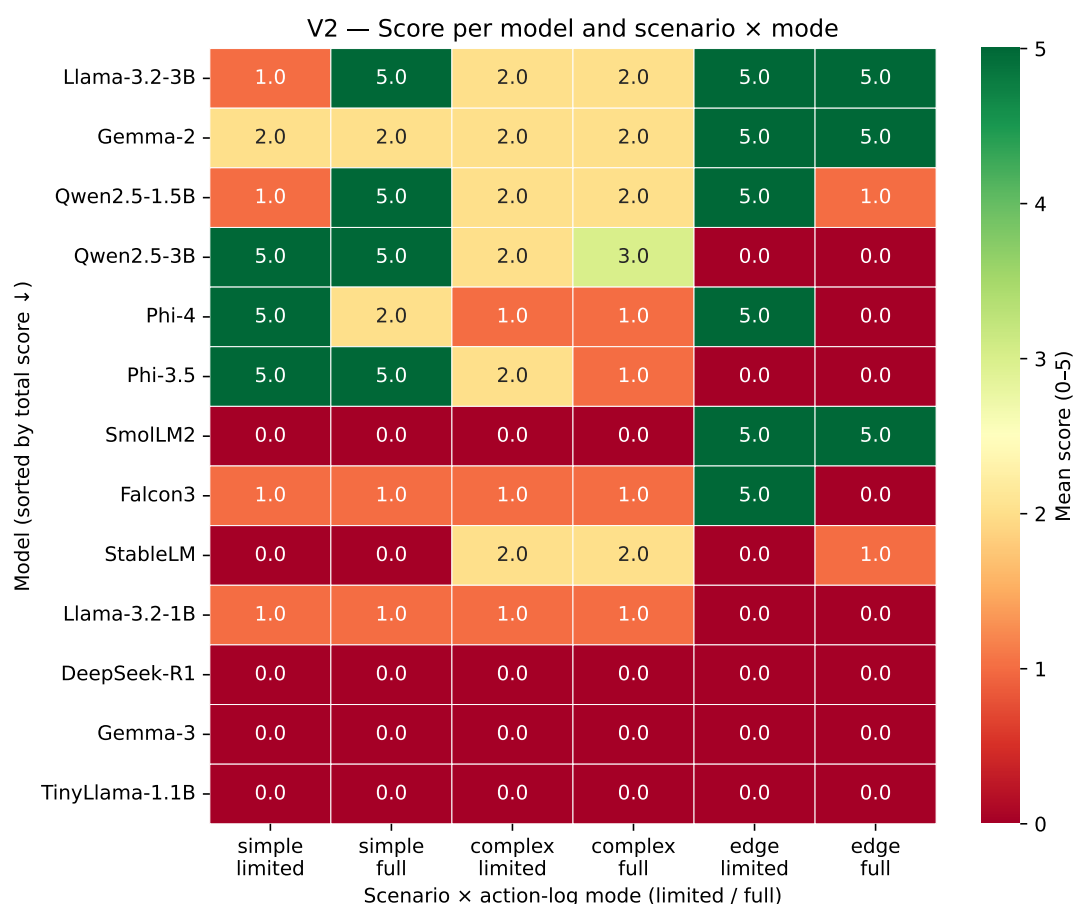


Figura 6.2: Score por modelo, escenario y modo de log (V2).

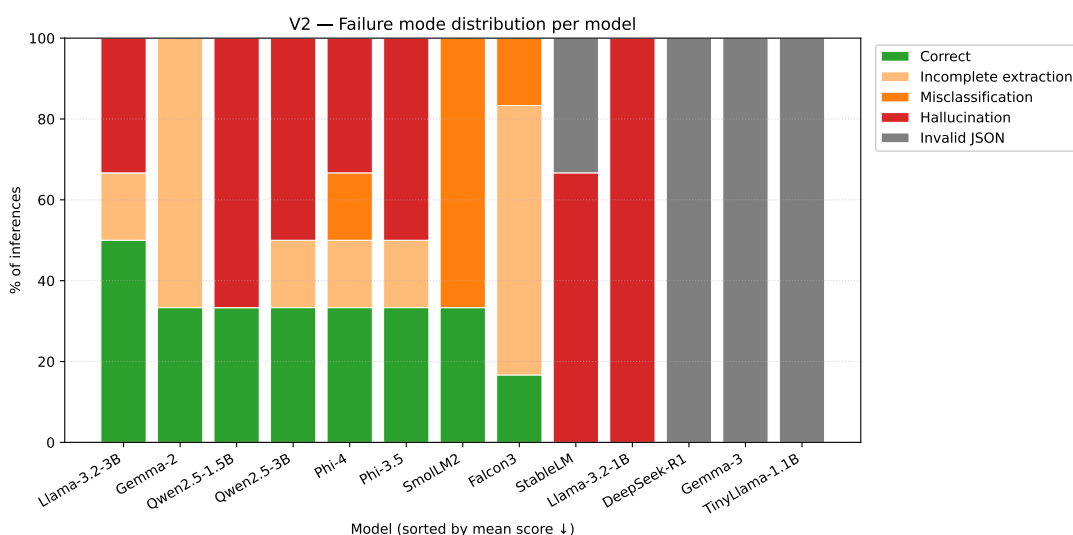


Figura 6.3: Distribución de modos de fallo por modelo (V2).

De V2 se concluye que el tamaño mínimo viable para producir JSON consistente ronda los 1,5 B de parámetros, que el modo del log debe ajustarse por modelo (se adopta *limited* por defecto, que no perjudica a ningún modelo) y que el ranking, con prompts sintéticos, sitúa a

Llama-3.2-3B como mejor modelo general.

6.5. Validación con prompt real (V3)

Los prompts sintéticos de V2 difieren sustancialmente del prompt real que genera el demostrador con `perception_mode:=real`: este contiene ocho detecciones YOLO con campos de confianza y ubicación (parte de ellos null), *guidelines* explícitas de interpretación dentro del propio prompt, y la traza completa de las acciones `move/pick_book` recientes, alcanzando unos 3 050 *tokens*. V3 captura ese prompt real y lo usa como entrada fija para los 10 modelos supervivientes de V2, con `temperature = 0.0` y tres iteraciones.

Pos.	Modelo	Score	Latencia mediana	VRAM
1	Qwen2.5-1.5B	5.00	2 002 ms	~1,0 GB
1	Qwen2.5-3B	5.00	3 911 ms	~2,0 GB
1	Phi-4-mini	5.00	5 230 ms	~2,5 GB
1	Phi-3.5-mini	5.00	5 650 ms	~2,5 GB
5	Llama-3.2-3B	1.00	4 059 ms	~2,0 GB
5	Gemma-2-2B	1.00	3 957 ms	~1,5 GB
5	StableLM-3B	1.00	6 141 ms	~2,0 GB
5	Llama-3.2-1B	1.00	1 781 ms	~0,8 GB
9	SmolLM2-1.7B	0.00	2 130 ms	~1,1 GB
9	Falcon3-1B	0.00	2 360 ms	~0,7 GB

Tabla 6.5: Ranking V3

El resultado central de V3 es la **inversión del ranking**: Llama-3.2-3B, mejor modelo en V2, produce con el prompt real un par `remove == add` constante, un no-op semántico que no resuelve nada. Los únicos modelos que procesan correctamente las ocho observaciones YOLO y respetan las *guidelines* son los Qwen2.5 y los Phi. La Figura 6.4 conecta el *score* V2 con el V3 por modelo y evidencia visualmente la inversión (Phi-3.5 +2,83; Phi-4 +2,67; Qwen2.5-3B +2,50; frente a Llama-3.2-3B -2,33; Gemma-2 -2,00). La Figura 6.5 muestra la frontera de Pareto V3: cuatro modelos empatan a 5/5 en el caso *simple* con el *prompt* real capturado, por lo que la frontera **degenera a un único óptimo, Qwen2.5-1.5B**, que domina al resto en latencia y memoria.

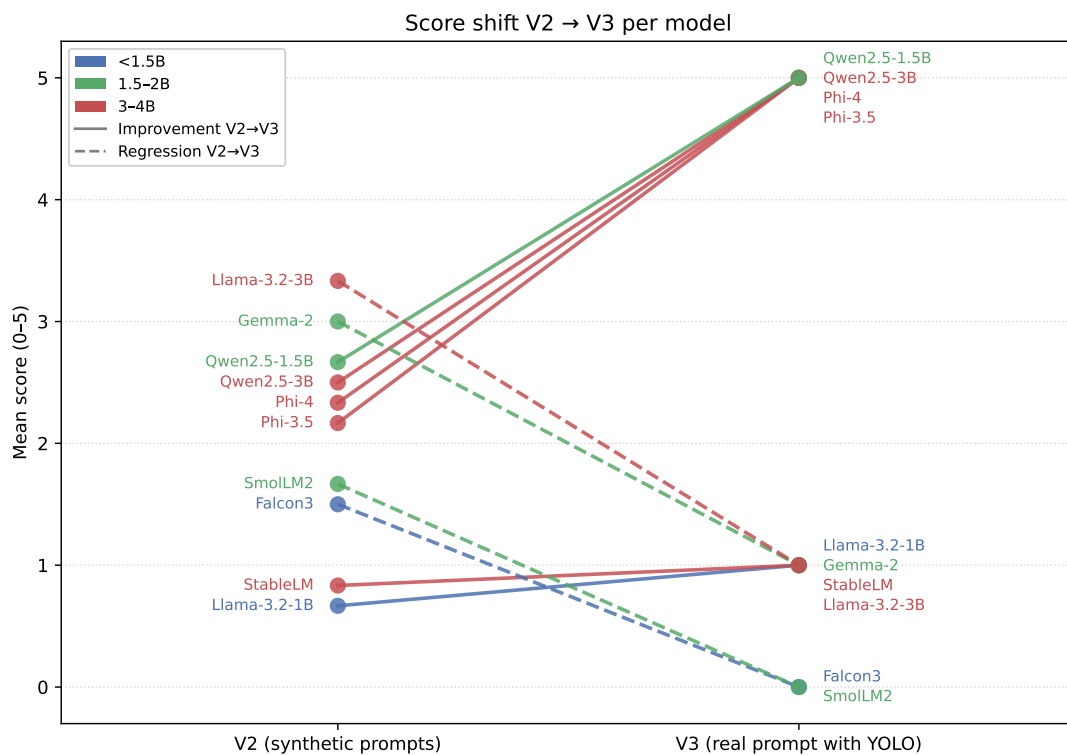


Figura 6.4: Cambio de score V2 a V3 por modelo.

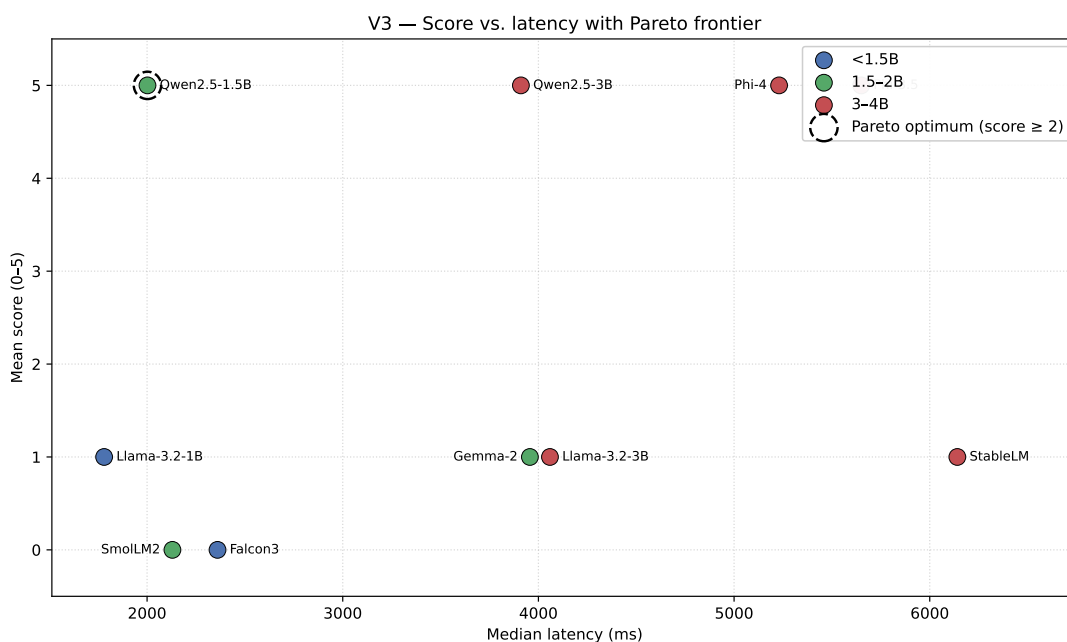


Figura 6.5: Frontera de Pareto con el prompt real (V3).

La conclusión de V3 es que una comparativa basada en prompts sintéticos no predice el comportamiento con datos reales, y que la elección final para hardware con GPU es **Qwen2.5-1.5B**: 5/5 en el caso simple con *prompt* real, ~2 s de latencia y ~1 GB de VRAM.

6.6. Despliegue sin GPU

Para evaluar el despliegue en hardware sin GPU se repitió V3 con los 10 modelos en configuración solo CPU (sin *offload*, sin *flash attention*, 4 hilos), midiendo sobre el i7-12700H limitado a 4 hilos como aproximación de límite inferior. Los resultados se resumen en la Tabla 6.6.

Modelo	Score GPU	Score CPU	Lat. GPU	Lat. CPU	Ratio
Phi-3.5-mini	5.00	5.00 (+3 alucin.)	5 638 ms	41 653 ms	×7,4
Phi-4-mini	5.00	5.00	5 331 ms	15 834 ms	×3,0
Qwen2.5-3B	5.00	5.00	3 817 ms	11 424 ms	×3,0
Qwen2.5-1.5B	5.00	1.00 (no-op)	2 005 ms	6 894 ms	×3,4
Llama-3.2-3B	1.00	1.00	3 932 ms	11 811 ms	×3,0
Gemma-2-2B	1.00	1.00	3 846 ms	12 471 ms	×3,2
StableLM-3B	1.00	1.00	6 208 ms	29 468 ms	×4,7
Llama-3.2-1B	1.00	1.00	1 790 ms	5 633 ms	×3,1
SmolLM2-1.7B	0.00	0.00	2 136 ms	11 257 ms	×5,3
Falcon3-1B	0.00	0.00	2 358 ms	6 489 ms	×2,8

Tabla 6.6: GPU frente a CPU en V3

Ocho de los diez modelos mantienen su *score* al cambiar de *backend*: la capacidad del modelo no depende del backend. La excepción crítica es **Qwen2.5-1.5B**, que colapsa de 5/5 a 1/5 en CPU para el caso *simple* con *prompt* real produciendo el mismo no-op que Llama-3.2-3B. La causa es la decodificación voraz con `temperature = 0.0`: las pequeñas diferencias numéricas entre los kernels CUDA y CPU cambian el *token* elegido en algún paso clave, y en un modelo de 1,5 B el margen entre la opción correcta y la alternativa es estrecho, por lo que la divergencia se materializa. Además, **Phi-3.5 mantiene 5/5 pero introduce tres alucinaciones constantes en CPU**, lo que en producción modificaría el estado PDDL más allá de lo necesario. Las Figuras 6.6 y 6.7 visualizan estos dos hallazgos.

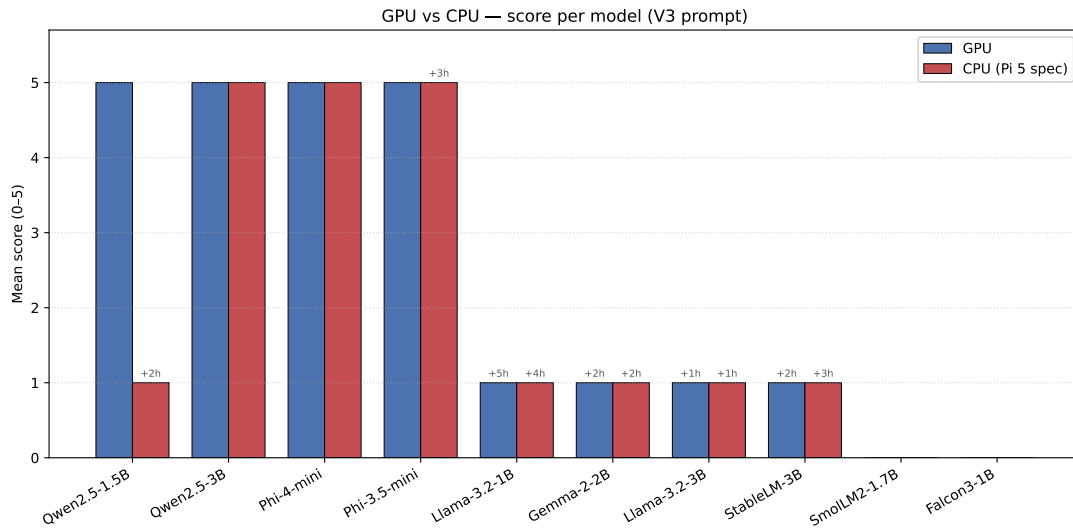


Figura 6.6: Score GPU frente a CPU (V3).

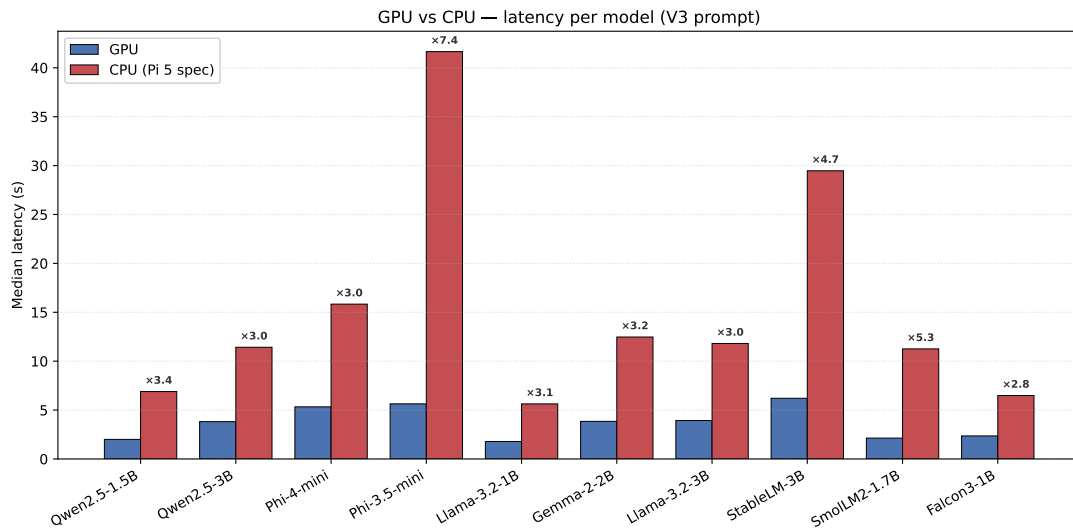


Figura 6.7: Latencia GPU frente a CPU (V3).

De aquí se deriva una recomendación bifurcada: en hardware con GPU, Qwen2.5-1.5B; en hardware solo CPU, **Qwen2.5-3B**, el modelo más rápido entre los que mantienen 5/5 limpio sin GPU —Phi-4-mini también alcanza 5/5 pero con un 28 % más de latencia (11,4 s vs 15,8 s), y Phi-3.5 introduce alucinaciones constantes.

6.7. Viabilidad en Raspberry Pi 5

La medición sobre i7 limitado a 4 hilos es un límite inferior optimista de la Pi 5 real. Para obtener un dato firme se ejecutó el modelo recomendado para CPU, Qwen2.5-3B-Q4_K_M, sobre una Raspberry Pi 5 de 16 GB (SoC Broadcom BCM2712, Cortex-A76 *quad-core* a 2,4 GHz, sin

AVX), con el prompt real del demostrador ($\sim 3\,050$ *tokens*) y refrigeración activa verificada (`vcgencmd get_throttled = 0x0`, sin *throttling*).

Caracterización base (sin optimización). Una inferencia completa tarda **226 s** (3 min 46 s), de los que **168 s (74 %)** corresponden al procesamiento del prompt y el resto a la generación (~ 115 *tokens* a $1,99$ *tokens/s*). El pico de memoria residente es de 4,06 GB, holgado en 16 GB. La salida es funcionalmente correcta (JSON bien formado, clasificación y predicados correctos contra el *ground truth*, sin alucinaciones). El factor frente a GPU es de $\times 58$, y frente al i7 a 4 hilos de $\times 20$: limitar el número de hilos en x86 captura la disponibilidad de paralelismo pero no las diferencias de IPC, frecuencia e ISA (ausencia de vectorización ancha en ARM). Como caracterización base, el módulo es **funcionalmente viable pero inviable para replanificación en tiempo real** en Pi 5.

6.8. Reutilización de KV-cache: validación empírica del contrato de orden

La caracterización anterior identifica el procesamiento del prompt como el cuello de botella sobre hardware embarcado: 168 s de 226 s, esto es, el 74 % del coste *wall-clock*. Esta sección valida la hipótesis de diseño introducida en la Sección 5.4.4 —situar el contenido invariante al principio del prompt y el contenido volátil al final— como una contribución de ingeniería medible, no como una elección estética. La medición se realiza sobre la propia Raspberry Pi 5, con el *patch* de reutilización de KV-cache contribuido a `llama_ros` (Sección 5.4.5) y los parámetros `pre_launch=true` y `cache_prompt=true` activos.

La Figura 6.8 reproduce tres invocaciones consecutivas del módulo sobre el mismo modelo y la misma sesión persistente. La primera llamada parte en frío (3 391 *tokens* de prompt) y consume 199 s, de los que 156,3 s corresponden a procesamiento del prompt y el resto a generación. La segunda llamada repite el prompt completo: el procesamiento se reduce a 0,49 s —el servidor identifica el prefijo idéntico, lo reutiliza desde la caché y sólo evalúa la última posición—. El coste total cae a 43 s, una aceleración de $\times 4,6$. La tercera llamada modifica únicamente los 27 *tokens* finales del prompt (el sufijo dinámico, correspondiente a un evento adicional añadido al final del resumen del log): el coste de procesamiento sube a 3,7 s y el total a 48 s. Este último caso es la evidencia más directa del contrato de orden: una modificación pequeña al final del prompt sólo recomputa el sufijo, mientras que la cabecera constante, el dominio PDDL y el problema vigente —la mayor parte de los *tokens*— se reutilizan íntegramente.

Esta ganancia transforma un coste de *prompt-eval* prohibitivo (156 s por llamada) en uno marginal (< 4 s) y hace viable un bucle de replanificación reactiva sobre hardware embarcado real. La contribución, además, no es privada del TFG: ha sido aceptada en el proyecto `llama_ros upstream` (PR n.º 38, commit 7b93280), lo que la hace verificable y reutilizable independientemente

de este trabajo.

Sobre el banco i7 con GPU, el mismo *patch* reutiliza correctamente 2 960 de 2 961 tokens del prefijo común, lo que confirma su funcionamiento. Sin embargo, el impacto en *wall-clock* sobre GPU es marginal: el procesamiento del prompt representa una fracción mucho menor del tiempo total cuando la inferencia se acelera por kernels CUDA. Esto subraya que la ganancia relevante de esta contribución es precisamente sobre el hardware embarcado, donde el procesamiento del prompt domina.

La diferencia entre la caracterización base (226 s, 168 s de prompt) y la llamada en frío con el *patch* (199 s, 156 s de prompt) corresponde a corridas distintas: difieren en el recuento de *tokens* del prompt, en la longitud de la salida y en la propia reordenación del prompt, que ya reduce ligeramente el coste en frío. La aceleración relevante es la de la reutilización en caliente ($\times 4.6$), que transforma un coste de prompt prohibitivo en uno marginal y hace viable un bucle de replanificación sobre Pi 5.

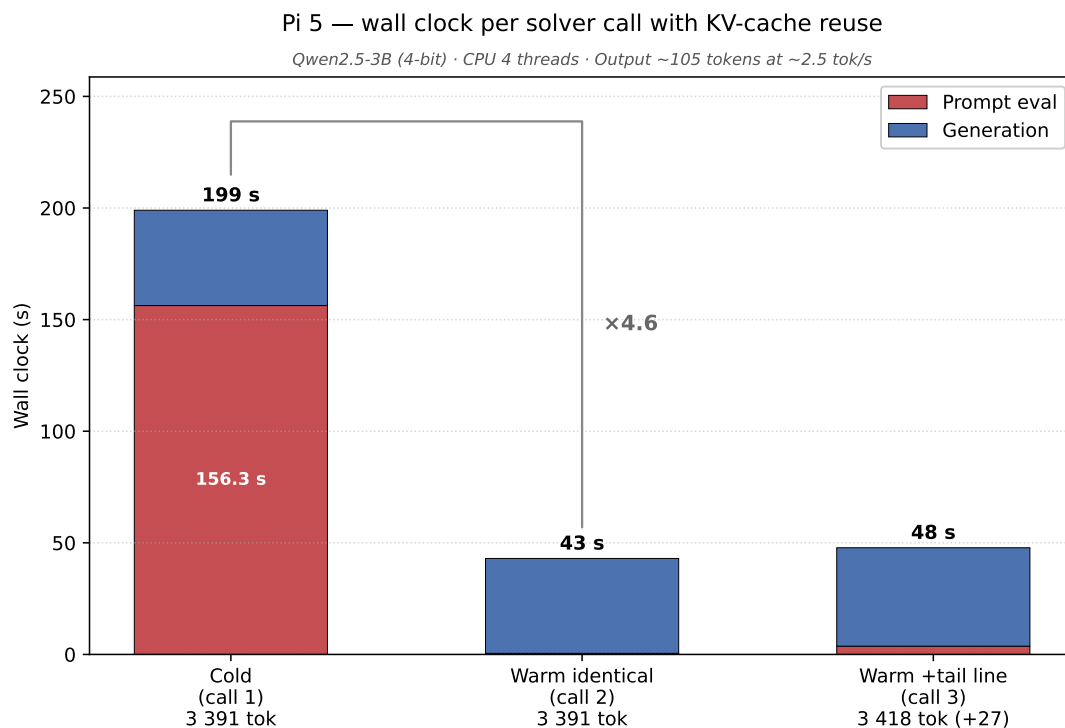


Figura 6.8: Reutilización de KV-cache en Raspberry Pi 5.

6.9. Limitaciones de la validación

La validación realizada tiene límites que deben especificarse para no extrapolar de forma injustificada. La comparativa de modelos se basa en un único caso de uso (*bookstore_kobuki*) y un *ground truth* concreto; un dominio distinto podría reordenar el ranking. Además, la combinación *prompt* real $\times$ caso *complex* no se ha evaluado: el demostrador real corresponde a un caso

simple. Las conclusiones sobre el escenario **complex** proceden de V2, con *prompts* sintéticos, y dado que este trabajo muestra que el comportamiento sintético no predice necesariamente el real (Figura 6.4), deben interpretarse como indicativas y quedan como trabajo futuro. El determinismo con `temperature = 0.0` elimina la varianza estocástica pero introduce sensibilidad numérica al *backend* (caso Qwen2.5-1.5B en CPU), por lo que los resultados de un modelo no son trasladables entre hardware sin re-medición. La medición en Raspberry Pi 5 se realizó con una sola iteración por el coste temporal de cada inferencia, suficiente para una caracterización de viabilidad pero no para un análisis estadístico. Finalmente, las latencias en CPU del banco i7 son un límite inferior, no una predicción de la Pi 5, como evidencia la diferencia de $\times 7-9$ entre la estimación previa y la medida real.

Capítulo 7

Impacto y responsabilidades

Este capítulo recoge el análisis de impacto y responsabilidad asociado al sistema desarrollado, tal como exige la memoria del Trabajo Fin de Grado. Se abordan cuatro dimensiones: el impacto social derivado de incorporar una capa de adaptación basada en modelos de lenguaje a un sistema robótico, el impacto económico de un módulo distribuido como software libre y ejecutado sobre hardware de bajo coste, el impacto medioambiental asociado al uso de inferencia local sobre modelos pequeños, y la responsabilidad ética y profesional inherente al uso de un componente no determinista dentro de un sistema que actúa sobre el mundo físico.

7.1. Impacto social

El trabajo aporta a los sistemas robóticos una capa de adaptación ante situaciones no previstas: cuando el mundo real no coincide con el estado simbólico, el sistema puede recuperarse sin intervención humana, lo que mejora la autonomía en tareas de servicio. El riesgo social de delegar interpretación en un modelo de lenguaje, como decisiones opacas o incorrectas, se mitiga por diseño: el modelo nunca decide ni actúa directamente, sino que propone, y toda propuesta pasa por las capas de validación descritas en la Sección 5.6 antes de poder influir en el comportamiento del robot. La trazabilidad de cada decisión, incluida la conservación de la respuesta cruda del modelo en el campo `resolution` del mensaje del monitor, permite auditar a posteriori el origen de cualquier cambio de estado.

7.2. Impacto económico

El módulo se distribuye como software libre bajo licencia Apache-2.0 y se integra con herramientas abiertas como PlanSys2, ROS 2 y `llama_ros`, por lo que incorpora capacidades de replanificación a un sistema existente sin desarrollar uno completo desde cero ni adquirir licen-

cias. La decisión de usar un modelo pequeño ejecutado localmente elimina el coste por consulta de los servicios de inferencia remotos y la dependencia de un proveedor externo. La validación de viabilidad sobre Raspberry Pi 5 del Capítulo 6 muestra que la reutilización de KV-cache hace el sistema utilizable sobre hardware de bajo coste, lo que reduce la barrera económica de desplegar replanificación deliberativa en robots de servicio.

7.3. Impacto medioambiental

Este trabajo no entrena modelos: la huella de cómputo se limita a inferencia sobre modelos pequeños cuantizados. La caracterización energética está acotada y medida: una inferencia base en Raspberry Pi 5 consume del orden de 226 s de cómputo, reducidos a un coste marginal en las llamadas en caliente gracias a la reutilización de KV-cache analizada en el Capítulo 6. Frente al uso de modelos grandes o servicios remotos, la elección de un modelo de entre 1,5 y 3 B de parámetros ejecutado localmente reduce sustancialmente el consumo por consulta y evita el tráfico de red asociado.

7.4. Responsabilidad ética y profesional

El uso de modelos de lenguaje en un sistema que actúa sobre el mundo físico exige que sus salidas no se incorporen de forma ciega. La arquitectura propuesta materializa este principio mediante validación previa, trazabilidad de las decisiones y separación entre propuesta, validación y ejecución. En lo relativo a propiedad intelectual, se han respetado las licencias de las dependencias utilizadas; el código desarrollado se publica bajo Apache-2.0 y la mejora realizada sobre `llama_ros` se ha devuelto al proyecto original como contribución abierta y aceptada *upstream*, descrita en la Sección 5.4.5, en coherencia con la práctica profesional de no privatizar mejoras sobre software libre del que se depende.

7.5. Uso responsable de LLMs en sistemas robóticos

En este trabajo el modelo de lenguaje desempeña un papel estrictamente auxiliar: interpreta contexto y propone hipótesis de actualización, pero no genera el plan, no decide qué acción ejecuta el robot y no modifica el estado por sí mismo. La separación entre propuesta, validación y ejecución, sostenida por mecanismos explícitos y verificables, es la condición que hace responsable el uso de un componente no determinista dentro de un sistema que debe ser consistente y trazable. El Capítulo 6 cuantifica además los modos de fallo del modelo, de modo que el riesgo no se asume de forma cualitativa sino medido.

Capítulo 8

Conclusiones y trabajo futuro

Este capítulo cierra los objetivos definidos en el Capítulo 2 y relaciona los requisitos iniciales con las evidencias de diseño, implementación y experimentación. La arquitectura modular de la Figura 4.1, los bloques funcionales de la Tabla 5.1, la construcción del *prompt* y la gestión de KV-cache del Capítulo 5, y los resultados del Capítulo 6 constituyen la base para valorar el alcance real del trabajo.

8.1. Consecución de objetivos

El objetivo general —diseñar, implementar y validar un módulo complementario, independiente y verificable de asistencia a la replanificación mediante modelos de lenguaje sin modificar PlanSys2— se considera cumplido con matices: el módulo existe, compila y se integra como capa externa (Capítulos 4 y 5, Figura 4.1), y su comportamiento se valida empíricamente en el Capítulo 6. Las garantías que el monitor aislado no provee —validación semántica completa y aceptación operativa— se delegan explícitamente en el nodo integrador, lo que se refleja como cumplimiento parcial de OE7 en la Tabla 8.1 y de RF-05 en la Tabla 8.2.

Elemento	Evidencia en la memoria	Estado
OE1: analizar ROS 2, PlanSys2 y planificación	Capítulo 3	Cumplido
OE2: arquitectura modular con responsabilidades separadas	Figura 4.1; Tabla 5.1	Cumplido
OE3: submódulo independiente y compilable	Capítulo 5; Apéndice A	Cumplido
OE4: interfaz cliente/servidor verificable	Sección 4.4.1; Capítulo 6	Cumplido
OE5: <i>backends</i> LLM vía <i>plugins</i>	Diseño con <code>pluginlib</code> ; contribución a <code>llama_ros</code>	Parcial: demostrado con un <i>backend</i>
OE6: contexto y <i>prompts</i> estructurados	Sección 5.4; Listing 5.4	Cumplido
OE7: procesar y validar respuestas	Secciones 5.5 y 5.6	Parcial: estructural en monitor, semántica/aceptación en integrador
OE8: escenario robótico y comparativa de modelos	Capítulo 6	Cumplido

Tabla 8.1: Objetivos y evidencias

Elemento	Evidencia en la memoria	Estado
RF-01: integración ROS 2 + PlanSys2	Capítulo 4; demostrador del Capítulo 6	Cumplido
RF-02: recopilar estado/eventos/contexto	Bloque B1 y resumen del registro de acciones	Cumplido
RF-03: construir <i>prompts</i> estructurados	Listing 5.4	Cumplido
RF-04: procesar respuesta del LLM	Extracción JSON tolerante en Sección 5.5.1	Cumplido
RF-05: validar antes de replanificar	Sección 5.6	Parcial: estructural en módulo; resto en integrador
RF-06: escenarios reproducibles	Capítulo 6; repositorios del Apéndice A	Cumplido
RNF-01: no modificar PlanSys2	Sección 4	Cumplido
RNF-02: componentes diferenciados	Tabla 5.1	Cumplido
RNF-03: instalación reproducible	Repositorios y documentación del proyecto	Cumplido
RNF-04: Jazzy/Rolling	Capítulos 5 y 6	Validado en Jazzy; Rolling soportado por diseño
RNF-05: trazabilidad	Campo <code>resolution</code> y <code>prompt_debug</code>	Cumplido
RNF-06: fallos e inconsistencias	Sección 5.7	Cumplido

Tabla 8.2: Requisitos y evidencias

8.2. Conclusiones técnicas

El trabajo entrega un módulo de replanificación asistida por LLM reutilizable e integrable con PlanSys2 sin modificar su núcleo (Capítulos 4 y 5, Figura 4.1), validado experimentalmente sobre un demostrador robótico real (Capítulo 6). De esa validación se extraen las siguientes conclusiones técnicas.

La primera conclusión demostrada es que, para la tarea acotada de corrección de estado PDDL del demostrador, un modelo de lenguaje pequeño local de 1,5–3 B de parámetros cuantizado es suficiente: Qwen2.5-1.5B, Qwen2.5-3B y los modelos Phi alcanzan 5/5 en el caso *simple* con el *prompt* real capturado del demostrador (Tabla 6.5), sin reentrenamiento ni inferencia remota. El alcance y los límites de esta afirmación se acotan en las Secciones 8.3 y 8.4.

La segunda conclusión es metodológica: una comparativa con *prompts* sintéticos no predice necesariamente el comportamiento con datos reales. La Figura 6.4 muestra la inversión entre V2 y V3: modelos bien posicionados con entradas sintéticas degradan al recibir el *prompt* real, mientras que otros mejoran. Por ello, la evaluación de componentes LLM integrados en robótica debe incluir entradas capturadas del flujo real del sistema.

La tercera conclusión se refiere al despliegue. En hardware embarcado, el coste dominante no es la generación sino el procesamiento del *prompt*. La Figura 6.8 muestra que la reutilización de KV-cache reduce el coste de una segunda llamada con prefijo común de 199 s a 43 s, y convierte el procesamiento del prefijo compartido en un coste marginal. Esta evidencia valida el contrato de ordenación estático-primero/dinámico-último definido en la implementación.

La cuarta conclusión es que la elección de modelo depende del *backend* y del hardware. La comparativa GPU/CPU de la Tabla 6.6 y las Figuras 6.6 y 6.7 muestran que Qwen2.5-1.5B es la mejor opción con GPU, pero degrada en CPU; en ejecución sin GPU, Qwen2.5-3B resulta más estable. La recomendación de modelo, por tanto, no debe trasladarse entre plataformas sin re-medición.

La quinta conclusión es arquitectónica: la separación entre propuesta, validación y ejecución permite introducir un componente no determinista sin delegarle el control del plan. El LLM genera hipótesis de actualización; el sistema conserva la responsabilidad de comprobarlas y decidir si afectan al problema de planificación.

Una última contribución de ingeniería con validación externa es la mejora de reutilización de KV-cache desarrollada para este trabajo e integrada en el proyecto *llama_ros upstream* (PR #38, fusionado; Sección 5.4.5, Apéndice A). No es una mejora privada del TFG: ha sido revisada y aceptada por el proyecto original, lo que la hace verificable de forma independiente.

8.3. Limitaciones del sistema

El sistema presenta limitaciones que deben enunciarse con precisión. La validación se ha realizado sobre un único caso de uso y un único *ground truth*; un dominio distinto podría reordenar el ranking de modelos. El uso de decodificación determinista (`temperature = 0.0`) elimina la varianza estocástica, pero introduce sensibilidad numérica al *backend*, como evidencia el colapso de Qwen2.5-1.5B al pasar de GPU a CPU. El despliegue sin aceleración en Raspberry Pi 5 es funcionalmente correcto, pero no apto para replanificación en tiempo real sin reutilización de KV-cache. Por último, de los tres niveles de validación descritos, solo el estructural se implementa dentro del módulo; la validación semántica y la decisión de aceptación se delegan en la aplicación integradora. Estas limitaciones delimitan el alcance de las conclusiones y orientan las líneas de trabajo futuro.

8.4. Validez y escalabilidad del enfoque local

La hipótesis central validada en este trabajo es que, para una tarea acotada de corrección de estado PDDL, un modelo de lenguaje pequeño local de 1,5–3 B de parámetros puede ser suficiente si se combina con un *prompt* restrictivo, salida JSON y validación externa. Esto define un punto de operación reproducible, económico y sin dependencia de red.

No obstante, los resultados también muestran límites claros: ningún modelo evaluado resuelve de forma robusta el escenario `complex` en V2 (Figuras 6.2 y 6.3), la combinación *prompt* real $\times$ caso `complex` queda pendiente de validación y algunos modelos pequeños presentan *no-op* semántico, alucinaciones o sensibilidad numérica al *backend*. Es razonable plantear como línea futura la comparación con modelos de mayor capacidad, incluidas familias de frontera accesibles vía API, para evaluar si reducen estos modos de fallo. Esa hipótesis debe validarse experimentalmente y no se considera un resultado de este TFG.

La adopción de modelos remotos introduciría además contrapartidas que el enfoque local evita por diseño: coste por llamada, dependencia de un proveedor externo, latencia y disponibilidad de red, imposibilidad de operar sin conectividad y posible entrega de datos a terceros. La arquitectura basada en `pluginlib` permite incorporar un *backend* de frontera como plugin adicional y compararlo bajo las mismas métricas, lo que constituye una línea futura natural.

8.5. Competencias del grado aplicadas

El trabajo ha aplicado competencias de programación de sistemas (C++ y Python sobre ROS 2), arquitectura software (diseño modular basado en *plugins*, separación de responsabilidades,

gestión de errores y concurrencia), planificación automática (PDDL, dominio/problema, integración con un planificador POPF), integración de sistemas robóticos (navegación, percepción, árboles de comportamiento y planificación sobre una misma pila), metodología experimental (diseño incremental de una comparativa, métricas, análisis de modos de fallo y autocrítica de la validez) y documentación técnica.

8.6. Nuevas competencias adquiridas

El TFG ha permitido adquirir competencias no cubiertas explícitamente por el plan de estudios: despliegue e inferencia de modelos de lenguaje sobre hardware con recursos limitados, diseño de *prompts* como contrato de rendimiento frente a la caché de un modelo autorregresivo, evaluación comparativa rigurosa de modelos pequeños, y contribución a un proyecto de software libre de terceros mediante la integración de una mejora *upstream* aceptada en su rama principal.

8.7. Líneas de trabajo futuro

Como trabajo futuro se plantea ampliar el número de dominios y escenarios evaluados, mejorar los mecanismos de validación semántica, estudiar técnicas de reducción de contexto, añadir soporte para diferentes *backends* LLM y explorar la integración del módulo con otros sistemas de planificación o ejecución robótica. También resulta relevante extender la validación a la tarea robótica completa de extremo a extremo, comparar el enfoque local con un segundo *backend* LLM —por ejemplo, un proveedor remoto vía API— y validar el flujo completo sobre ROS 2 Rolling. Estas líneas futuras derivan directamente de las limitaciones observadas: complejidad del escenario *complex*, dependencia del hardware, alcance limitado del *ground truth* y validación semántica delegada.

Bibliografía

- [1] Open Robotics. «ROS 2 Documentation,» visitado 6 de mayo de 2026. dirección: <https://docs.ros.org/>
- [2] PlanSys2 contributors. «PlanSys2 Documentation and Repository,» visitado 6 de mayo de 2026. dirección: https://github.com/PlanSys2/ros2_planning_system
- [3] A. J. Coles, A. I. Coles, M. Fox y D. Long, «Forward-Chaining Partial-Order Planning,» en *Proceedings of the Twentieth International Conference on Automated Planning and Scheduling*, 2010.
- [4] Qwen Team, «Qwen2.5 Technical Report,» *arXiv preprint arXiv:2412.15115*, 2024.
- [5] D. McDermott et al., «PDDL: The Planning Domain Definition Language,» en *AIPS-98 Planning Competition Committee*, 1998.
- [6] International Planning Competition. «International Planning Competition,» visitado 17 de mayo de 2026. dirección: <https://www.icaps-conference.org/competitions/>
- [7] B. Liu et al., «LLM+P: Empowering Large Language Models with Optimal Planning Proficiency,» *arXiv preprint arXiv:2304.11477*, 2023.
- [8] M. Ahn, A. Brohan, N. Brown et al., «Do As I Can, Not As I Say: Grounding Language in Robotic Affordances,» en *Proceedings of the 6th Conference on Robot Learning*, 2022.
- [9] I. Singh et al., «ProgPrompt: Generating Situated Robot Task Plans using Large Language Models,» en *Proceedings of the IEEE International Conference on Robotics and Automation*, 2023.
- [10] K. Lin, C. Agia, T. Migimatsu, M. Pavone y J. Bohg, «Text2Motion: From Natural Language Instructions to Feasible Plans,» en *Proceedings of Robotics: Science and Systems*, 2023.
- [11] G. Gerganov y contributors. «llama.cpp,» visitado 6 de mayo de 2026. dirección: <https://github.com/ggml-org/llama.cpp>
- [12] M. González-Santamarta y contributors. «llama_ros: ROS 2 bindings for llama.cpp,» visitado 21 de mayo de 2026. dirección: https://github.com/mgonzs13/llama_ros

- [13] E. Frantar, S. Ashkboos, T. Hoefler y D. Alistarh, «GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers,» *arXiv preprint arXiv:2210.17323*, 2022.
- [14] J. Lin, J. Tang, H. Tang, S. Yang, X. Chen y S. Han, «AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration,» *Proceedings of Machine Learning and Systems*, 2024.
- [15] Z. Ji et al., «Survey of Hallucination in Natural Language Generation,» *ACM Computing Surveys*, 2023.
- [16] K. Valmeekam, M. Marquez, S. Sreedharan y S. Kambhampati, «On the Planning Abilities of Large Language Models: A Critical Investigation,» *arXiv preprint arXiv:2305.15771*, 2023.
- [17] S. Kambhampati et al., «LLMs Can't Plan, But Can Help Planning in LLM-Modulo Frameworks,» *arXiv preprint arXiv:2402.01817*, 2024.

Apéndice A

Repositorio del proyecto

Este anexo recoge la información necesaria para localizar el código desarrollado durante el trabajo. La Tabla A.1 recoge los repositorios públicos del proyecto y la rama de referencia.

Elemento	Valor
Organización del proyecto	https://github.com/plansys2-llm
Repositorio del módulo	https://github.com/plansys2-llm/plansys2_llm_monitor
Repositorio del demostrador	https://github.com/plansys2-llm/plansys2_llm_examples
Rama	main
Licencia	Apache-2.0
ROS 2 testeado	Jazzy validado; Rolling soportado por diseño
Commit de referencia	Revisión final del TFG
Contribución <i>upstream</i>	llama_ros PR #38, fusionado en 7b93280

Tabla A.1: Repositorios del proyecto

El proyecto se distribuye en dos repositorios complementarios bajo la organización plansys2-llm: plansys2_llm_monitor, que contiene el módulo de replanificación, y plansys2_llm_examples, que contiene el demostrador plan_bookstore. Ambos se publican bajo licencia Apache-2.0. Además, la mejora de reutilización de KV-cache se integró en llama_ros mediante la PR #38, como se describe en la Sección 5.4.5.